

Awarding Body:	Arden University
Programme Name:	Master of Science – Data Science
Module Name (and Part if applicable):	Programming for Data Analytics (COM7024)
Assessment Title:	Project
Student Number:	
Tutor Name:	
Word Count:	3427
Please refer to the Word Count Policy on your Module Page for guidance	

Table of Contents

<i>Cover Page</i>	<i>Error! Bookmark not defined.</i>
<i>Table of Contents</i>	2
<i>List of Figures</i>	3
<i>Introduction</i>	4
<i>Methodology</i>	5
1.0 Data Preparation	5
Stage 1: Initial Dataset Exploration (EDA).....	5
Stage 2: Data Preprocessing	10
Stage 3: Dataset Validation.....	32
2.0 Data Analysis and Visualization	33
3.0 Reporting	36
1. Interpretation of Visuals and Justification.....	36
2. Key Findings and Recommendations.....	43
3. Limitations.....	44
<i>Reflective Report</i>	45
<i>References</i>	47
<i>Appendix</i>	49

List of Figures

Figure 1: Importing necessary libraries.....	5
Figure 2: Dataset content overview.....	6
Figure 3: Dataset structure overview.....	7
Figure 4: Initial statistical summary.....	7
Figure 5: Proof of duplicates and missing data.....	8
Figure 6: Evidence of deduplication.....	10
Figure 7: Evidence of listwise deletion after deduplication.....	11
Figure 8: Evidence of handling Incorrect Formatting.....	12
Figure 9: Creation of new data frame for investigation.....	14
Figure 10: Check to verify if living area columns are the same.....	14
Figure 11: Summary of investigation of gap.....	15
Figure 12: Investigating the relationship between gap and renovation status.....	15
Figure 13: Diagrammatic representation of results.....	16
Figure 14: Implementation of Correlation Analysis.....	17
Figure 15: Pearson correlation heatmap.....	19
Figure 16: Removal of 'Living Area (sqft) 2' column.....	19
Figure 17: Logical consistency checks.....	21
Figure 18: Calculation of excess from violations.....	21
Figure 19: Evidence of handling incorrect data.....	22
Figure 20: Calculating the distribution of data.....	23
Figure 21: Code for plotting shape of data.....	24
Figure 22: Shape of Price, Total Area, and Living Area variables.....	25
Figure 23: Calculating outliers using 1.5 IQR.....	26
Figure 24: Code for visualizing outlier ranges.....	27
Figure 25: Visual representation of outliers.....	27
Figure 26: Investigation of rows containing Price outliers.....	28
Figure 27: Investigation of rows containing Living Area outliers.....	29
Figure 28: Investigation of rows containing Total Area outliers.....	30
Figure 29: Evidence of measurement unit conversion.....	31
Figure 30: Final data transformation.....	32
Figure 31: Dataset exploration after preprocessing.....	32
Figure 32: Code snippet for analysis and visualization 1.....	33
Figure 33: Code snippet for analysis and visualization 2.....	34
Figure 34: Visual results from analysis.....	35
Figure 35: Key metrics summary.....	36
Figure 36: Price vs Living Area.....	37
Figure 37: Price vs Total Area.....	38
Figure 38: Price vs Year Built.....	38
Figure 39: Price vs Property Grade.....	39
Figure 40: Price distribution with luxury threshold.....	41
Figure 41: Price distribution by Renovation Status.....	42
Figure 42: Proof of claims in (2).....	44

Introduction

Nowadays, the metaphor “data is the new oil” applies to many industries, including real estate. A typical data analysis project follows a workflow known as the data analysis lifecycle (Sharma & Garg, 2024), which includes problem definition, data collection, preparation, analysis, and reporting. This project begins after data collection, as a problem definition and dataset have already been provided containing data on housing in Berlin. This project entails assessing data quality, addressing any found issues using appropriate preprocessing techniques, followed by exploratory data analysis (EDA) and communicating insights as part of my role as a junior data analyst at a German real estate company. Emphasis is laid on investigating the distribution of property pricing and total property size, as well as the relationship between living area and living square footage.

Following the data analysis lifecycle, this project starts with an assessment of the dataset's structure and quality, followed by preprocessing to make it ready for analysis. Statistical summaries and visualizations will be used to identify patterns, trends, and relationships within the data. The findings shall be presented alongside practical recommendations to support the data analyst manager in understanding housing market characteristics in Berlin and informing future predictive work.

Methodology

Referencing the data analysis lifecycle, this section entails ensuring that findings from the analysis are accurate, reproducible, and free of bias.

1.0 Data Preparation

This refers to making raw data usable for analysis, encompassing data preprocessing and EDA. Data preprocessing refers to cleaning data, while EDA involves getting acquainted with the characteristics of a dataset. In data cleaning, it is essential to understand what unclean data is; this is data that fits into one or more of the following:

- Data containing empty/missing values
- Data containing incorrect values and/or outliers
- Data in the wrong format
- Duplicate data

Data preparation will be carried out in three stages:

Stage 1: Initial Dataset Exploration

To explore this dataset, it must first be uploaded, followed by importing the necessary libraries for interacting with it, such as Pandas for data manipulation and Matplotlib and Seaborn for analysis and visualization.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Figure 1: Importing necessary libraries

Table 0: Summary of the Dataset

Attribute	Value
Dataset name	BerlinHousing4049.csv
Number of observations (records/rows)	9999
Number of variables (fields/columns)	11
Numerical (quantitative) variables	11
Categorical (qualitative) variables	0
Missing values	62
Duplicate records	2

```
df = pd.read_csv('/content/BerlinHousing4049.csv')
df
```

	price	bedrooms	bathrooms	sqft_living	sqft_total	floors	condition	grade	built	renovated	living_area_sqft
0	221900.0	3.0	1.00	NaN	5650.0	1.0	3.0	7	1955.0	0	1340.0
1	538000.0	3.0	NaN	2570.0	7242.0	2.0	3.0	7	1951.0	1991	1690.0
2	180000.0	2.0	1.00	NaN	10000.0	1.0	3.0	6	1933.0	0	NaN
3	604000.0	4.0	3.00	1960.0	5000.0	1.0	5.0	7	1965.0	0	1360.0
4	510000.0	3.0	2.00	1680.0	8080.0	1.0	3.0	8	1987.0	0	NaN
...	...	...	...	...	...	...	...	...	...	...	...
9994	1695000.0	4.0	2.75	3770.0	10900.0	2.0	5.0	9	1924.0	0	3000.0
9995	279000.0	2.0	1.00	1020.0	47044.0	1.0	5.0	7	1904.0	1958	1930.0
9996	262000.0	3.0	1.50	1700.0	9579.0	1.0	4.0	7	1962.0	0	1700.0
9997	533000.0	4.0	1.00	1550.0	4750.0	1.5	3.0	7	1919.0	0	1320.0
9998	640000.0	3.0	1.75	1680.0	8100.0	1.0	3.0	8	1950.0	0	1880.0

9999 rows x 11 columns

Figure 2: Dataset content overview

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9999 entries, 0 to 9998
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype  
---  ---
0   price                9990 non-null   float64
1   bedrooms             9988 non-null   float64
2   bathrooms            9990 non-null   float64
3   sqft_living          9989 non-null   float64
4   sqft_total           9993 non-null   float64
5   floors               9999 non-null   float64
6   condition            9998 non-null   float64
7   grade                9999 non-null   int64  
8   built                9996 non-null   float64
9   renovated            9999 non-null   int64  
10  living_area_sqft     9986 non-null   float64
dtypes: float64(9), int64(2)
memory usage: 859.4 KB
```

Figure 3: Dataset structure overview

Figure 3 shows that all variables were imported as numeric types, just as in the original dataset. However, several columns, such as *price*, *bedrooms*, *built*, etc., were imported as *float64* data type rather than integer. This occurred because the variables contained missing values, causing Pandas to use a floating-point representation called *NaN* (*Not a Number*) to accommodate them (Pandas Contributors, 2025). Correcting the data types for these variables will be handled in the next stage.

	price	bedrooms	bathrooms	sqft_living	sqft_total	floors	condition	grade	built	renovated	living_area_sqft
count	9.990000e+03	9988.000000	9990.000000	9989.000000	9.993000e+03	9999.000000	9998.000000	9999.000000	9996.000000	9999.000000	9986.000000
mean	5.333292e+05	3.364638	2.063263	2052.985985	1.604135e+04	1.432043	3.446389	7.594459	1967.267307	91.642364	1975.878029
std	3.768880e+05	0.910976	0.765275	910.804284	4.505641e+04	0.511869	0.666341	1.165901	27.988813	417.830430	672.424849
min	7.500000e+04	1.000000	0.500000	380.000000	5.720000e+02	1.000000	1.000000	3.000000	1900.000000	0.000000	620.000000
25%	3.150000e+05	3.000000	1.500000	1410.000000	5.425000e+03	1.000000	3.000000	7.000000	1950.000000	0.000000	1490.000000
50%	4.458190e+05	3.000000	2.000000	1890.000000	7.916000e+03	1.000000	3.000000	7.000000	1969.000000	0.000000	1830.000000
75%	6.398000e+05	4.000000	2.500000	2500.000000	1.118000e+04	2.000000	4.000000	8.000000	1990.000000	0.000000	2340.000000
max	7.700000e+06	11.000000	8.000000	12050.000000	1.651359e+06	3.500000	5.000000	13.000000	2015.000000	2015.000000	5790.000000

Figure 4: Initial statistical summary

```
columns_with_missing_values = len(df.columns[df.isnull().any()])
print('Columns with missing data:', columns_with_missing_values)

rows_with_missing_values = df.isnull().any(axis=1).sum()
print('Rows with missing data:', rows_with_missing_values)

missing_values = df.isnull().sum().sum()
missing_values_percentage = (missing_values / df.shape[0]) * 100
print(f'Percentage of missing values: {round(missing_values_percentage, 2)}%')

missing_values_percentage_per_column = round((df.isnull().sum() / df.shape[0]) * 100, 2)
display(missing_values_percentage_per_column.sort_values(ascending=False))

duplicates = df.duplicated().sum()
print(f'\nDuplicates count:', duplicates)
```

Columns with missing data: 8
 Rows with missing data: 55
 Percentage of missing values: 0.62%

	0
living_area_sqft	0.13
bedrooms	0.11
sqft_living	0.10
bathrooms	0.09
price	0.09
sqft_total	0.06
built	0.03
condition	0.01
floors	0.00
grade	0.00
renovated	0.00

dtype: float64
 Duplicates count: 2

Figure 5: Proof of duplicates and missing data

Figures 3 - 5 serve as proof of Table 0. The *describe()* method is used to obtain descriptive statistics of the data, while the *info()* method shows 9999 observations across eleven fields with a total of 62 missing values across eight fields and 55 records, as well as two duplicates. The summary statistics indicate substantial variability in several variables, particularly in *price* and *sqft_total*, as the maximum values are considerably larger than the upper quartiles (75% row of Figure 4), suggesting the presence of outliers.

Table 1: Explanation of Variables

Variable	Description	Important Notes
Price	Property cost (in Euros)	Euros because Berlin is found in Germany, which uses the Euro.
Bedrooms	Bedrooms count	
Bathrooms	Bathrooms count	The presence of decimals in the bathroom column, like 4.5 and 2.25, does not indicate a data error but follows established valuation standards. (National Association of REALTORS®, 2020), (Fannie Mae, 2024)
Squarefoot Living (sqft_living)	Interior living space	Measured in square feet
Squarefoot Total (sqft_total)	Property size on ground (Land/Plot size)	Measured in square feet
Floors	Floors count	The presence of decimals in the floors column, just as in the bathrooms column, does not indicate data error but follows established valuation standards. (Indiana Department of Local Government Finance, 2021)
Condition	Property state rating	By state, it means the building's maintenance level and level of wear-and-tear.
Grade	Overall property quality	Determined during the construction process.
Built	Year constructed	
Renovated	Year renovated	0 means the property has not been renovated, while an actual year indicates the property has been renovated and its year of renovation
Living Area Squarefoot (living_area_sqft)	Living Area Size	A similar column exists (sqft_livng) and it is also measured in square feet.

Stage 2: Data Preprocessing

This stage is important because data in its raw state is often not usable for analysis, as it is sometimes messy, inconsistent, and incomplete, which can distort results. This dataset will be cleaned in alignment with the categories for which dirty data is classified as listed in section 1.0:

1. Handling Duplicate Data

Duplicate data is characterized by redundant records within a dataset. Given this purely numerical dataset, the presence of duplicates can skew statistical findings, leading to incorrect conclusions. Ultimately, the dataset is deduplicated to ensure its integrity before analysis (Hammer, et al., 2023), using the `drop_duplicates()` method.

```
print('Row count before duplicate removal:', df.shape[0])
df.drop_duplicates(inplace=True)
print('Row count after duplicate removal:', df.shape[0])
```



```
Row count before duplicate removal: 9999
Row count after duplicate removal: 9997
```

Figure 6: Evidence of deduplication

2. Handling Missing Data

The dataset contains a negligible proportion of missing observations (0.62%), as shown in Figure 5. Given the large sample size of 9997 records (after deduplication), removing observations with missing values using the `dropna()` method is considered preferable, as over 99% of data will be preserved, thus having an insignificant impact on the analysis. Since this project requires performing EDA, retaining complete cases (performing listwise deletion) is considered the appropriate strategy (Heymans & Twisk, 2022).

```
print('% missing values before Listwise Deletion: 0.62%')
print('Row count before Listwise Deletion:', df.shape[0])
df.dropna(inplace=True)
print('Row count after Deduplication & Listwise Deletion:', df.shape[0])
print(f'% missing values after Listwise Deletion: {round((df.isnull().sum().sum()
/ df.shape[0]) * 100, 2)}%')

% missing values before Listwise Deletion: 0.62%
Row count before Listwise Deletion: 9997
Row count after Deduplication & Listwise Deletion: 9942
% missing values after Listwise Deletion: 0.0%
```

Figure 7: Evidence of listwise deletion after deduplication

3. Handling Incorrect Formatting

After reviewing the dataset for formatting inconsistencies, two improvements were implemented to enhance data quality and readability. Firstly, several fields had the wrong data type, as explained in paragraph two of Stage 1. The variables will be converted into integers using the *astype()* method of Pandas, ensuring that they are represented as they originally were, making the data semantically correct (Wickham, 2014). Secondly, the dataset's field names are renamed using the *rename()* method to improve readability, as meaningful variable names make datasets easier to interpret, reduce ambiguity during analysis, and contribute to more reproducible analytical workflows (McKinney, 2022).

```
# 1. Changing data type
print('Data types transformation')
int_columns = [
    'price',
    'bedrooms',
    'sqft_living',
    'sqft_total',
    'condition',
    'built',
    'living_area_sqft',
]
for column in int_columns:
    df[column] = df[column].astype('int64')

display(df.dtypes)

#2. Renaming columns
print('\nColumn renaming\n')
df.rename(columns={
    'price': 'Price',
    'bedrooms': 'Number of Bedrooms',
    'bathrooms': 'Number of Bathrooms',
    'sqft_living': 'Living Area (sqft)',
    'sqft_total': 'Total Area (sqft)',
    'floors': 'Number of Floors',
    'condition': 'Property Condition',
    'grade': 'Property Grade',
    'built': 'Year Built',
    'renovated': 'Year Renovated',
    'living_area_sqft': 'Living Area (sqft) 2',
}, inplace=True)
df.columns
```

Data types transformation

	0
price	int64
bedrooms	int64
bathrooms	float64
sqft_living	int64
sqft_total	int64
floors	float64
condition	int64
grade	int64
built	int64
renovated	int64
living_area_sqft	int64

dtype: object

Column renaming

```
Index(['Price', 'Number of Bedrooms', 'Number of Bathrooms',
      'Living Area (sqft)', 'Total Area (sqft)', 'Number of Floors',
      'Property Condition', 'Property Grade', 'Year Built', 'Year Renovated',
      'Living Area (sqft) 2'],
      dtype='object')
```

Figure 8: Evidence of handling Incorrect Formatting

4. Handling Incorrect Data and Outliers

Data preprocessing goes beyond everything performed above. It also includes the identification and handling of incorrect data and outliers, each of which represents a distinct aspect of data quality with a significant impact on analysis.

- Incorrect data refers to observations that do not accurately represent the real-world entity and violate expected constraints.
- In contrast, (Bruce, et al., 2020) describe outliers as observations that differ substantially from most data points in a dataset and require investigation to determine whether they represent legitimate variation or data errors.

This dataset will be assessed using logical data consistency checks, followed by statistical and graphical techniques to detect potential errors and outliers. Two variables appear similar (Figure 8); therefore, an investigation would be conducted to determine whether they have a relationship or if the data is duplicated before the logical consistency checks are applied.

1. Handling Incorrect Data

- a. **Investigating similar columns:** A copy of the data frame is created using the *copy()* method containing the columns required for investigation.

```
living_area_columns = df[['Living Area (sqft)', 'Living Area (sqft) 2',
                        'Year Renovated']].copy()
living_area_columns
```

	Living Area (sqft)	Living Area (sqft) 2	Year Renovated
3	1960	1360	0
8	1780	1780	0
10	3560	2210	0
11	1160	1330	0
12	1430	1780	0
...	...	...	...
9994	3770	3000	0
9995	1020	1930	1958
9996	1700	1700	0
9997	1550	1320	0
9998	1680	1880	0

9942 rows x 3 columns

Figure 9: Creation of new data frame for investigation

```
# step 1: get the % of identical count
identical_count = (living_area_columns['Living Area (sqft)'] == living_area_columns['Living Area (sqft) 2']).sum()
differing_count = (living_area_columns['Living Area (sqft)'] != living_area_columns['Living Area (sqft) 2']).sum()

print(f'Identical count: {identical_count}')
print(f'Differing count: {differing_count}')
print(f'Percentage of Identical Count: {identical_count / len(living_area_columns) * 100:.2f}%')
```

Identical count: 1069
Differing count: 8873
Percentage of Identical Count: 10.75%

Figure 10: Check to verify if living area columns are the same

Figure 10 shows these columns are not fully identical; therefore, one of them cannot easily be eliminated. Irrespective of most rows being non-identical, knowledge about how far apart their non-identical values are is unknown. Finding this gap/difference will indicate how substantial the discrepancy is; thus, determining whether it is noise or a meaningful difference.

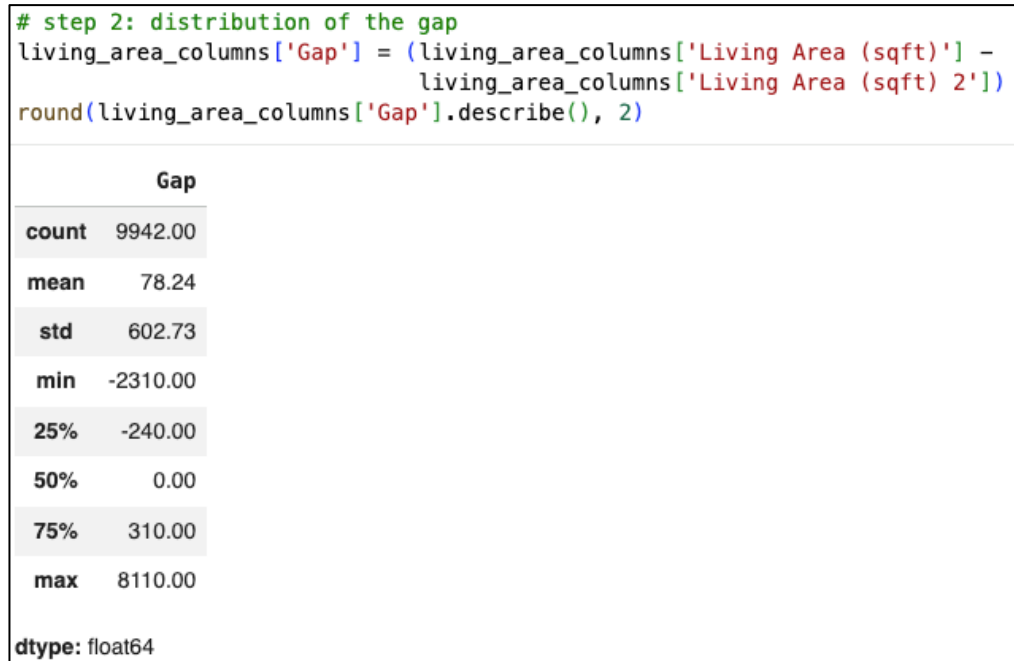


Figure 11: Summary of investigation of gap

Rather than evaluating the gap per record, the *describe()* method was used to summarize the difference. Figure 11 shows the gap averages ~78 square feet with minimum and maximum values of -2310 and 8110 square feet, respectively, indicating the difference is substantial and not noise: meaningless variation around a value that is fundamentally the same thing (Sharou, et al., 2021). Further investigation will be performed to determine whether a relationship exists between the gap and renovation status.

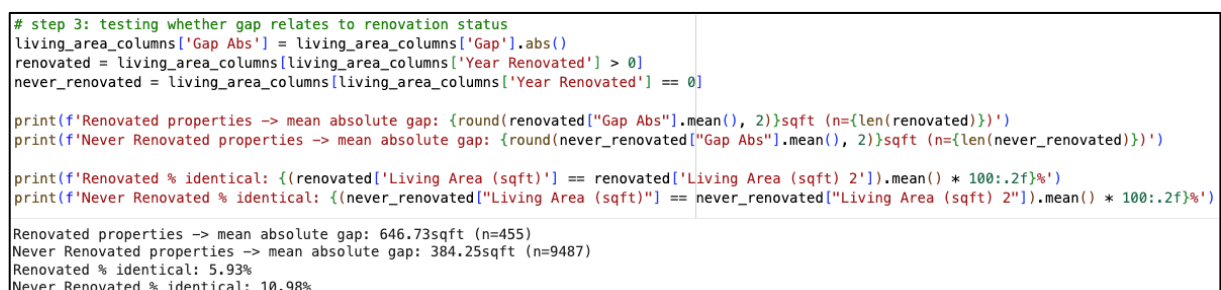


Figure 12: Investigating the relationship between gap and renovation status

According to Figure 12, the *abs()* method was used to prevent positive and negative gaps from cancelling out when calculating averages. For the 455 renovated properties, the average difference between the two living area columns is 646.73 square feet, and 384.24 square feet for the unrenovated ones. Also, 5.93% of renovated properties have both fields matching exactly, whereas it is 10.98% for never-renovated properties. This shows that renovation status has a relationship with the gap between both columns but does not determine it completely, as substantial gaps persist among never-renovated properties, indicating both columns likely reflect a distinction broader than renovation status alone.

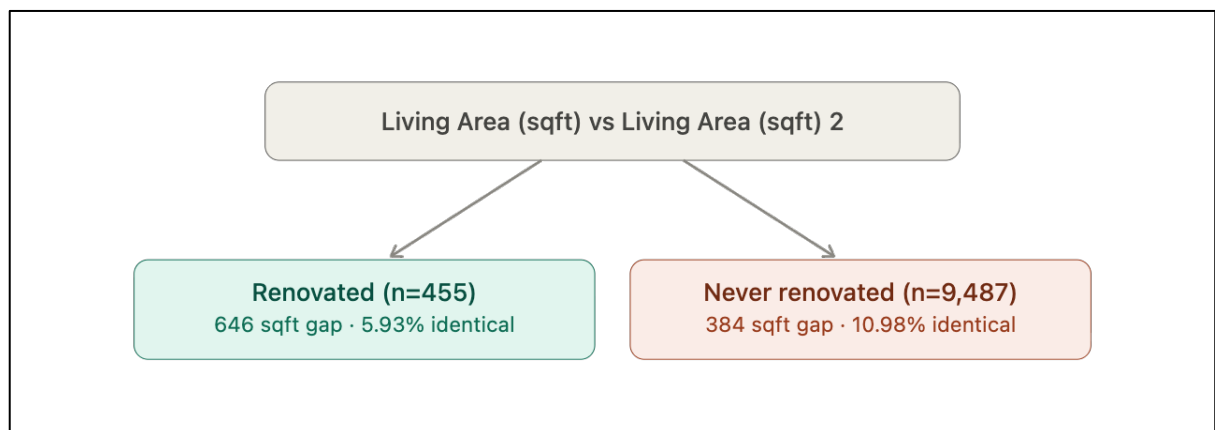


Figure 13: Diagrammatic representation of results

To statistically understand the relationship, correlation analysis is performed next.



Figure 14: Implementation of Correlation Analysis

Pearson's correlation method is used for statistical calculation of correlation alongside a heatmap for visual representation. Pearson's method was chosen because all fields are numerical and contain no missing values (Kent State University Libraries, 2026), while the heatmap was chosen because it features as a competing visual in a recent study comparing correlation visualization methods, particularly because of its color-code matrix (Dudáš, 2024), making it suitable for use here.

Pearson correlation coefficients are between -1 and +1 and are interpreted as follows:

Table 2: Interpretation of Pearson correlation coefficients

Sign	Meaning
+	Both variables move in the same direction; they either increase together or decrease together.
-	Also known as inverse correlation, it occurs when one variable increases while the other decreases.

Magnitude (correlation strength)	Meaning
0	No correlation
0 - 0.3	Weak correlation
0.3 - 0.6	Moderate correlation
0.6 - 0.8	Strong correlation
0.8 - 1.0	Very strong correlation

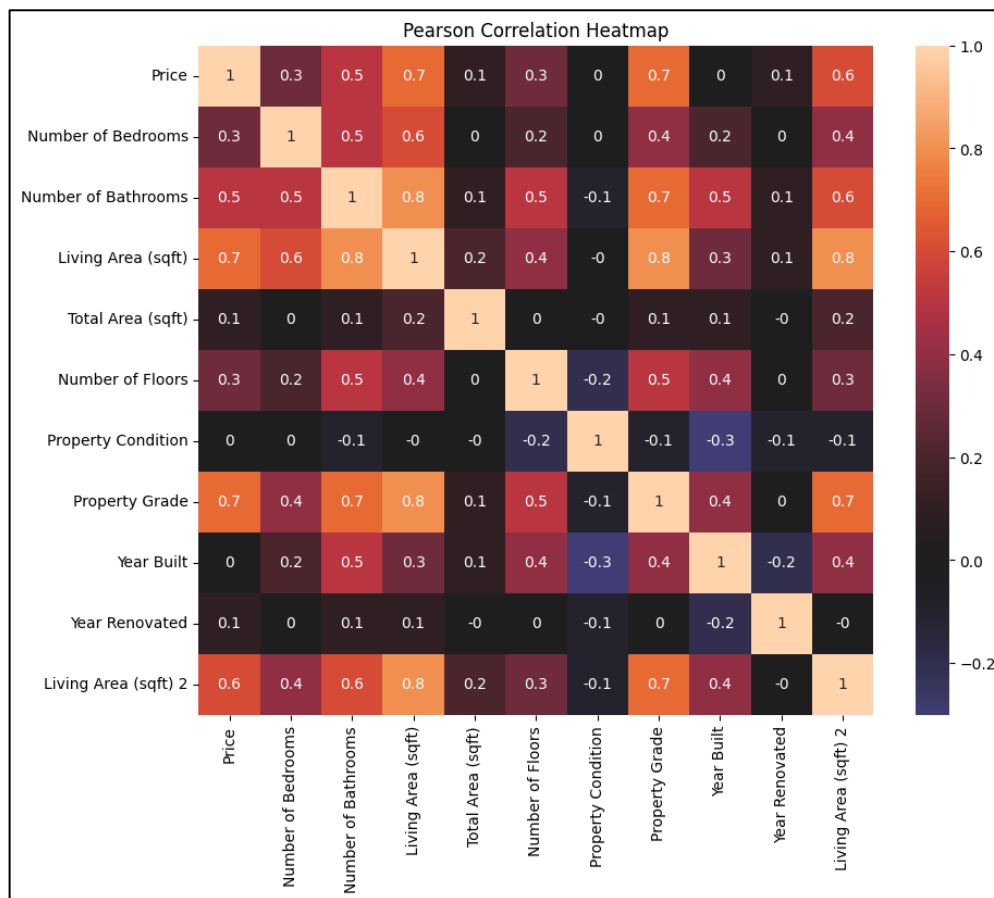


Figure 15: Pearson correlation heatmap

From Figure 15, *Living Area (sqft)* and *Living Area (sqft) 2* have a strong positive correlation, confirming the result of the investigation above; they are related ($r = 0.8$) but distinct. However, only one field will be used going forward, which is *Living Area (sqft)* because it correlates more with the dependent variable, *Price*, as shown above, making it more relevant to this analysis.

```
df.drop('Living Area (sqft) 2', axis=1, inplace=True)
df.columns

Index(['Price', 'Number of Bedrooms', 'Number of Bathrooms',
       'Living Area (sqft)', 'Total Area (sqft)', 'Number of Floors',
       'Property Condition', 'Property Grade', 'Year Built', 'Year Renovated'],
      dtype='object')
```

Figure 16: Removal of 'Living Area (sqft) 2' column

Figure 15 shows other strong relationships such as *Property Grade vs. Price* and *Living Area vs. Price*. At this level, this measures only their linear association and does not necessarily imply causation.

b. Checking for logical consistency

Table 3: Logical Consistency Check Rule

Variable	Validation Rule	Rationale
Price	> 0	Property prices cannot be negative or zero
Number of Bedrooms	≥ 0	Residential properties cannot have negative bedrooms
Number of Bathrooms	> 0	Residential properties cannot have zero or negative bathrooms
Number of Floors	> 0	A building must have at least one floor to physically exist
Total Area (sqft)	> 0	A property must occupy some measurable total area
Living Area (sqft)	> 0	The same as above. Zeroes or anything less is not valid for living area
Living Area (sqft)	$\leq \text{Total Area (sqft)} * \text{Number of Floors}$	The initial thought would be that the living area is a subset of total area and therefore should not exceed total area. However, this is not necessarily the case in this dataset. The assumption here is that if there is a property whose living area is more than its total area, then it means that the property has multiple floors. If it has just a single floor, then it is impossible for the living space to be more than the total space; thus, the given rule.
Year Renovated	$\geq \text{Year Built (when Year Renovated} \neq 0)$	A house cannot be renovated before it is built.
Year Renovated	$\leq \text{maximum value and} \geq \text{minimum value of the Year Built column}$	This serves as a soft check against the dataset's implied timeframe

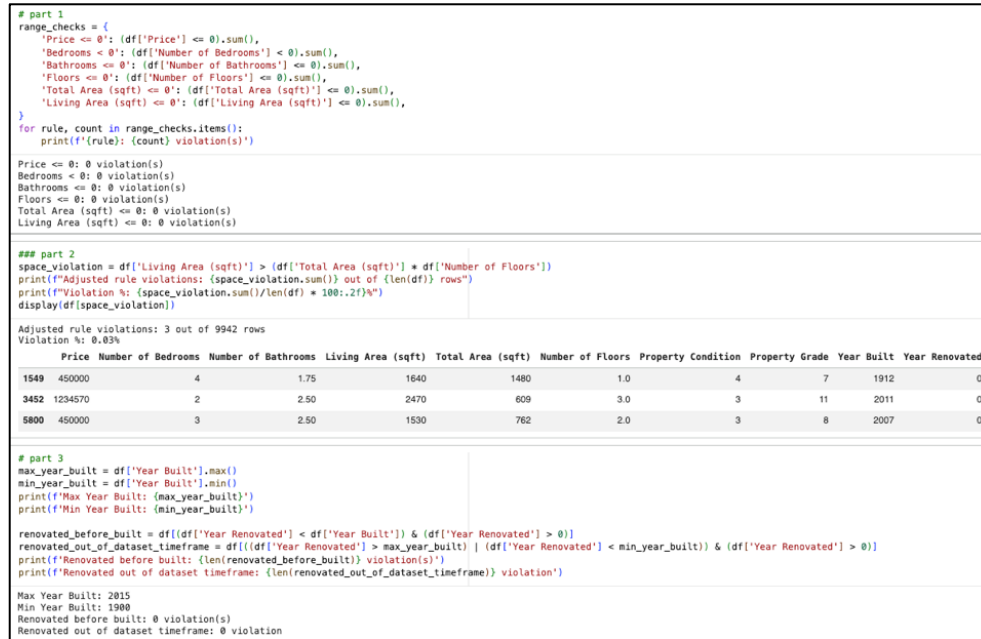


Figure 17: Logical consistency checks

Figure 17 shows that three rows constituting 0.03% of the data failed a validation check. The excess (difference between the living area and total area multiplied by the number of floors) would be calculated to determine which row(s) should be deleted.

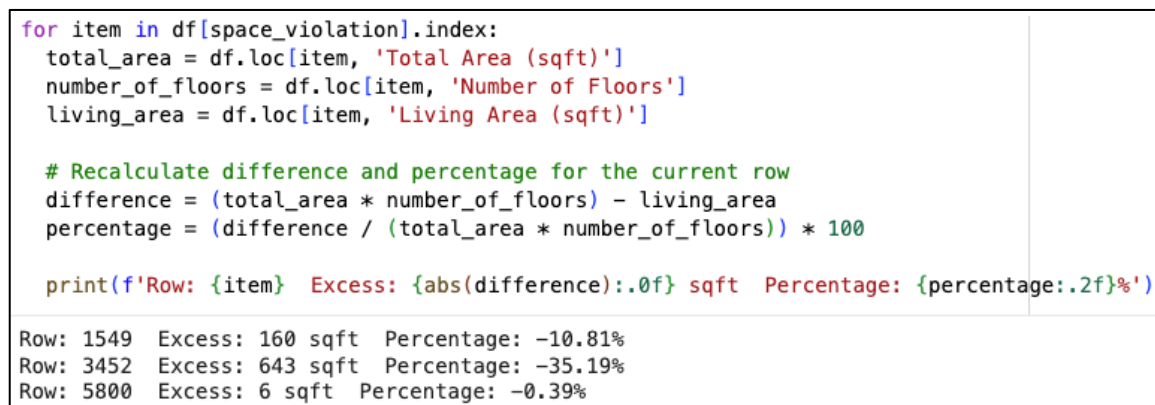


Figure 18: Calculation of excess from violations

As per Figure 18, row 5800 is negligible; therefore, treating it as an error would be too strict. The other rows would, however, be removed, as their margins are large.

```
df.drop(df[space_violation].index[0:2], inplace=True)
df.shape[0]
```

9940

Figure 19: Evidence of handling incorrect data

Finally, the selected rows are removed by specifying their positions using the *index()* method, and to verify the removal was successful, the *shape* attribute is indexed to get the row count as shown in Figure 19.

2. Handling Outliers

Before applying a method to check for outliers, it is standard practice to understand data distribution; its shape will inform the choice of the most appropriate method. To understand the distribution, values such as mean, mode, median, standard deviation, skewness, and kurtosis are calculated, followed by a graphical representation to visualize its shape.

```
# step 1: calculate distribution of data
variables = ['Price', 'Total Area (sqft)', 'Living Area (sqft)']
for variable in variables:
    print(f'----- {variable} -----')
    print(f'Mean: {round(df[variable].mean(), 2)}')
    print(f'Median: {round(df[variable].median(), 2)}')
    print(f'Mode: {round(df[variable].mode()[0], 2)}')
    print(f'Std. Dev: {round(df[variable].std(), 2)}')
    print(f'Skewness: {round(df[variable].skew(), 2)}')
    print(f'Kurtosis: {round(df[variable].kurtosis(), 2)}')
    print()

----- Price -----
Mean: 533721.15
Median: 446475.0
Mode: 350000
Std. Dev: 377251.6
Skewness: 5.01
Kurtosis: 53.42

----- Total Area (sqft) -----
Mean: 16052.87
Median: 7915.0
Mode: 5000
Std. Dev: 45147.06
Skewness: 14.68
Kurtosis: 340.99

----- Living Area (sqft) -----
Mean: 2054.34
Median: 1890.0
Mode: 1400
Std. Dev: 911.18
Skewness: 1.59
Kurtosis: 5.99
```

Figure 20: Calculating the distribution of data

These fields are included in the investigation because:

- Price: it is the dependent variable in this dataset and was emphasized in the project brief.
- Total Area: like Price, it was emphasized in the project brief.
- Living Area: it has a strong correlation with Price, unlike Total Area.

Figure 20 shows early on that the distribution is not normal because the mean, mode, and median are not nearly identical; this is the first indicator of skewness. Additionally, the skewness values for *Price* and *Total Area* are outside the acceptable range of -2 to +2, and the same applies for kurtosis of all variables (George & Mallery, 2019), which suggests the data will not have a normal (bell-curve) shape.

```
fig, axes = plt.subplots(len(variables), 1, figsize=(6, 4 * len(variables)))

for i, variable in enumerate(variables):
    sns.histplot(df[variable], bins=30, kde=True, ax=axes[i])
    axes[i].set_title(f'Histogram of {variable}')

plt.tight_layout()
plt.show()
```

Figure 21: Code for plotting shape of data

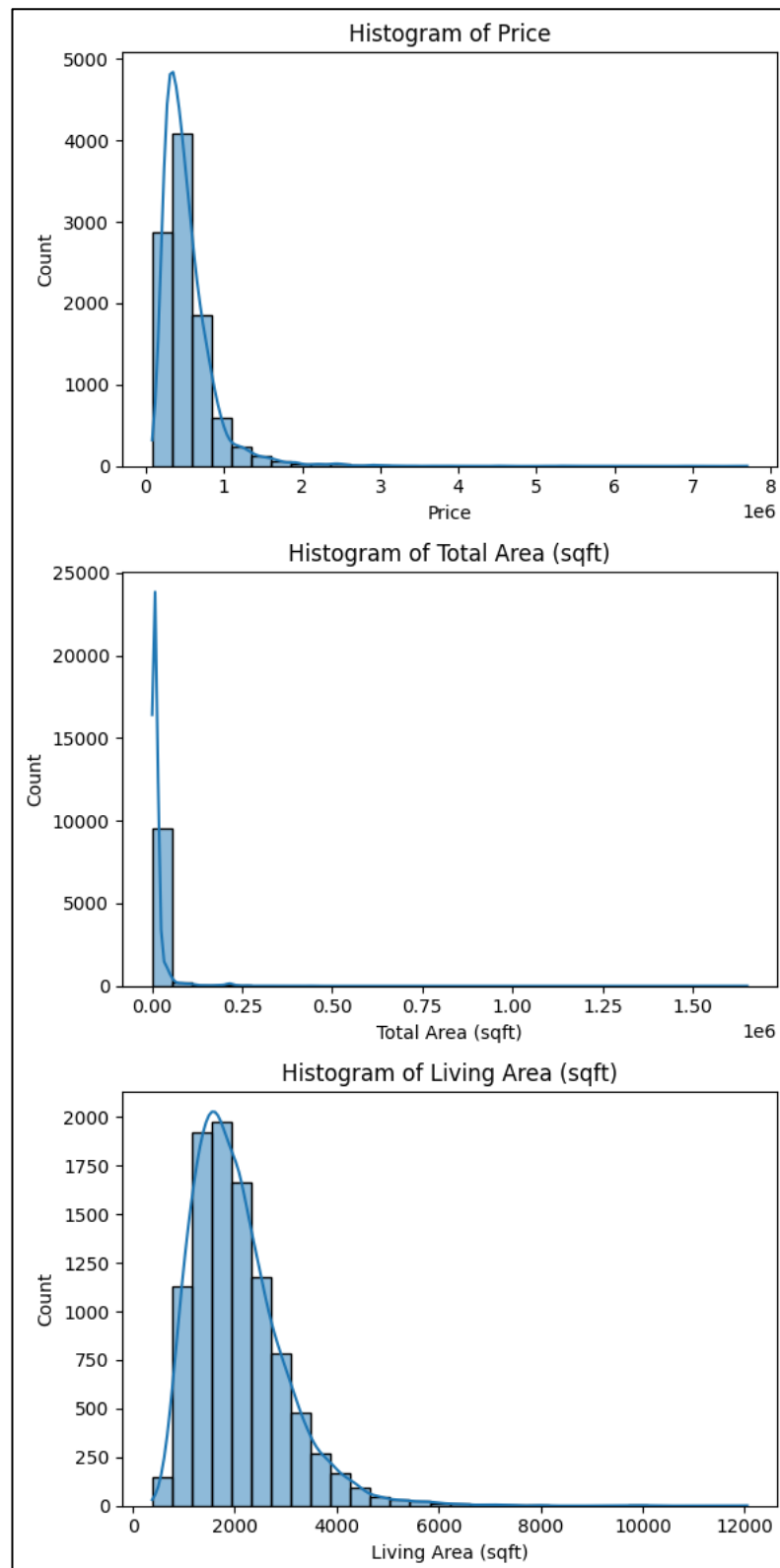


Figure 22: Shape of Price, Total Area, and Living Area variables

From the visuals above, it is evident that the data is not normally distributed. Therefore, the appropriate method for outlier detection is the 1.5 Interquartile Range (IQR) rule, as it does not require data to be normally distributed (Bruce, et al., 2020), unlike other methods such as Z-score.

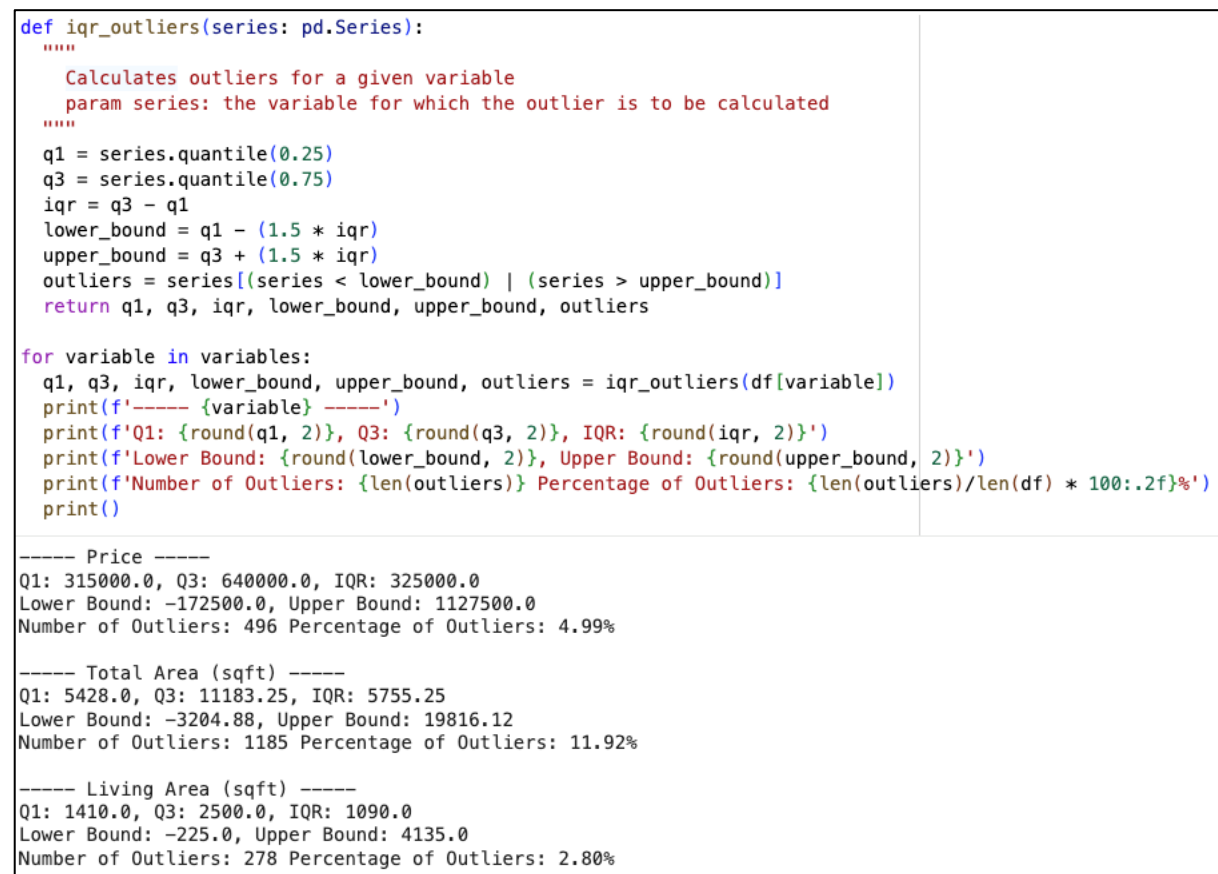


Figure 23: Calculating outliers using 1.5 IQR

According to Figure 23, the lower bound (fence) of all variables is negative, but this has no real-world meaning, as a property cannot cost or measure less than zero. Effectively, every outlier considered shall sit above the upper bound.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for i, variable in enumerate(variables):
    q1 = df[variable].quantile(0.25)
    q3 = df[variable].quantile(0.75)
    iqr = q3 - q1
    upper = q3 + 1.5 * iqr
    lower = q1 - 1.5 * iqr
    n_out = ((df[variable] < q1 - 1.5*iqr) | (df[variable] > upper)).sum()

    sns.boxplot(y=df[variable], ax=axes[i])
    axes[i].set_title(f'{variable}\nOutliers: {n_out} ({n_out/len(df)*100:.2f}%)', fontsize=10)
    axes[i].axhline(upper, color='red', linestyle='--', linewidth=1)
    axes[i].text(0.05, upper, f' Upper Fence: {upper:,.0f}', color='red', fontsize=8, va='bottom')
    axes[i].text(0.05, lower, f' Lower Fence: {lower:,.0f}', color='red', fontsize=8, va='bottom')

plt.tight_layout()
plt.show()
```

Figure 24: Code for visualizing outlier ranges

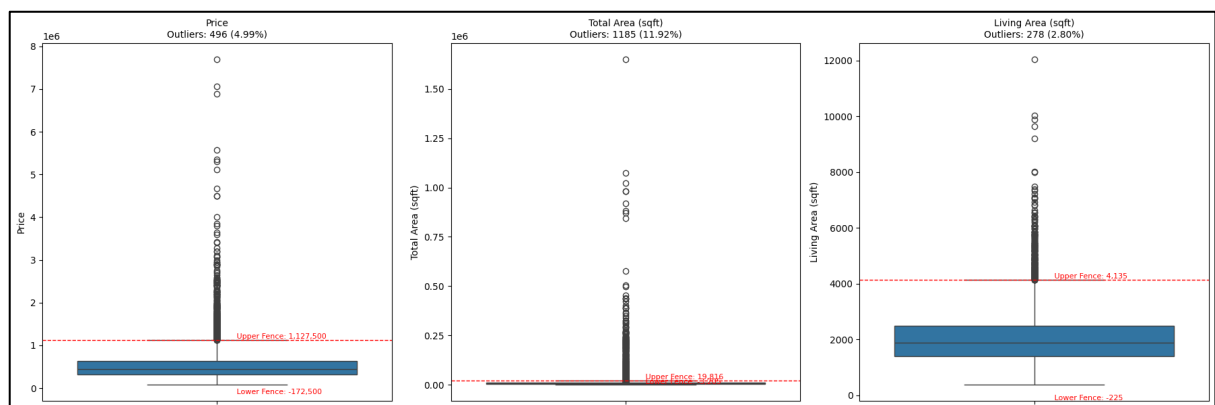


Figure 25: Visual representation of outliers

The boxplots in the visuals above use the same quartile logic as the IQR calculation, making it the best visual for representing outliers calculated using 1.5 IQR.

Now that outliers have been identified, the next step involves handling them; however, while outlier detection is a statistical step, outlier treatment requires domain knowledge. Thus, the question “Why?” (Why are some values unusually high?) must be answered for each variable to determine the correct action for handling them.

Table 4: Decision framework for handling outliers

Question	If answer is yes	If answer is no
Is the value extreme due to measurement or entry error?	Correct or remove	Continue to the next question
Is the value possible given real-world context, even if extreme?	Retain it, as removing it will bias the dataset towards “typical” cases only	Flag or remove

```
# Variable 1: Price
print('Rows containing outliers for the price column:')
price_outlier_rows = df.query('Price > 1127500')
display(price_outlier_rows.head(20).sort_values(by='Property Grade', ascending=True))
```

Rows containing outliers for the price column:

	Price	Number of Bedrooms	Number of Bathrooms	Living Area (sqft)	Total Area (sqft)	Number of Floors	Property Condition	Property Grade	Year Built	Year Renovated
125	1450000	4	2.75	2750	17789	1.5	3	8	1914	1992
69	1325000	5	2.25	3200	20158	1.0	3	8	1965	0
427	1300000	3	2.00	2350	15021	1.0	4	8	1976	0
49	1350000	3	2.50	2753	65005	1.0	5	9	1953	0
216	1505000	5	3.00	3300	33474	1.0	3	9	1957	1991
246	2400000	4	2.50	3650	8354	1.0	3	9	2000	0
282	2050000	5	3.00	3830	8480	2.0	5	9	1905	1994
21	2000000	3	2.75	3050	44867	1.0	3	9	1968	0
472	1199000	4	2.75	4110	8400	2.0	3	9	1928	2013
450	1450000	3	4.50	3970	24920	2.0	3	10	1977	1999
312	2384000	5	2.50	3650	9050	2.0	5	10	1921	0
300	3075000	4	5.00	4550	18641	1.0	3	10	2002	0
384	1350000	5	3.50	4800	14984	2.0	3	11	1998	0
269	2900000	4	3.25	5050	20100	1.5	3	11	1982	2008
313	1384000	4	3.25	4290	12103	1.0	3	11	1997	0
270	1365000	4	4.75	5310	57346	2.0	4	11	1989	0
314	1400000	4	3.25	4290	12103	1.0	3	11	1997	0
473	1150000	4	3.25	4190	10259	2.0	3	11	2000	0
419	1550000	5	4.25	6070	171626	2.0	3	12	1999	0
153	2250000	4	3.25	5180	19850	2.0	3	12	2006	0

Figure 26: Investigation of rows containing Price outliers

Price:

Figure 26 shows that houses with extreme prices represent a luxury segment; these houses have high values for property grade (minimum value = 8) and living area (typically includes bedrooms and bathrooms), which, as shown in Figure 15, have a strong correlation with pricing. Given that this luxury segment represents real-world context, these values will be retained. Moreover, the future objective of this project is to predict housing market trends, and removing approximately 5% (pricing outliers) of data works against it. Also, it would mean the dataset ends up containing only "typical" housing data without considering luxurious cases.

```
# variable 2: Living Area (sqft)
print('Rows containing outliers for the Living Area column:')
living_area_outlier_rows = df.query('`Living Area (sqft)` > 4135').sort_values('Living Area (sqft)', ascending=False)
display(living_area_outlier_rows.head(20))
```

Rows containing outliers for the Living Area column:										
	Price	Number of Bedrooms	Number of Bathrooms	Living Area (sqft)	Total Area (sqft)	Number of Floors	Property Condition	Property Grade	Year Built	Year Renovated
7252	7700000	6	8.00	12050	27600	2.5	4	13	1910	1987
3914	7062500	5	4.50	10040	37325	2.0	3	11	1940	2001
9254	6885000	6	7.75	9890	31374	2.0	3	13	2001	0
8092	4668000	5	6.75	9640	13068	1.0	3	12	1983	2009
4411	5570000	5	5.75	9200	35069	2.0	3	13	2001	0
1164	5110800	5	5.25	8010	45517	2.0	3	12	1999	0
1448	5350000	5	5.00	8000	23985	2.0	3	12	2009	0
4024	8000000	7	6.75	7480	41664	2.0	3	11	1953	0
1315	5300000	6	6.00	7390	24829	2.0	4	12	1991	0
2713	1110000	5	3.50	7350	12231	2.0	3	11	2001	0
6501	1940000	4	5.75	7220	223462	2.0	3	12	2000	0
7907	3200000	6	5.00	7100	18200	2.5	3	13	1933	2002
4149	4000000	4	5.50	7080	16573	2.0	3	12	2008	0
7035	3800000	5	5.50	7050	42840	1.0	4	13	1978	0
3020	2525000	4	5.50	6930	45100	1.0	4	11	1950	1991
2444	3278000	2	1.75	6840	10000	2.5	3	11	2001	0
4811	2479000	5	3.75	6810	7500	2.5	3	13	1922	0
2626	4500000	5	5.50	6640	40014	2.0	3	12	2004	0
1539	1300000	6	3.50	6563	32670	2.0	3	10	2002	0
3100	1500000	4	5.50	6550	217374	1.0	3	11	2006	0

Figure 27: Investigation of rows containing Living Area outliers

Living Area:

As per Figure 23, the living area column constitutes 2.8% of outliers, which is the smallest compared to the other fields. Figure 27 shows no evidence pointing to errors but rather shows the extreme values in this column are consistent with genuinely large properties (reference the *Price*, *Number of Bedrooms*, and *Number of Bathrooms* columns). Therefore, the outliers in this field will be retained.

<pre># Variable 3: Total Area (sqft) print('Rows containing outliers for the Total Area column:') total_area_outlier_rows = df.query('`Total Area (sqft)` > 19816.12').sort_values('Total Area (sqft)', ascending=False) display(total_area_outlier_rows.head(20))</pre>										
Rows containing outliers for the Total Area column:										
	Price	Number of Bedrooms	Number of Bathrooms	Living Area (sqft)	Total Area (sqft)	Number of Floors	Property Condition	Property Grade	Year Built	Year Renovated
1719	700000	4	1.00	1300	1651359	1.0	4	6	1920	0
7647	542500	5	3.25	3010	1074218	1.5	5	8	1931	0
7769	855000	4	3.50	4030	1024068	2.0	3	10	2006	0
3949	998000	4	3.25	3770	982998	2.0	3	10	1992	0
4441	790000	2	3.00	2560	982278	1.0	3	8	2004	0
6691	1998000	2	2.50	3900	920423	2.0	3	12	2009	0
7077	1650000	4	3.25	3920	881654	3.0	3	11	2002	0
9714	937500	4	4.00	5545	871200	2.0	3	11	2003	0
4540	550000	3	2.00	3650	843309	2.0	4	7	1991	0
7294	700000	2	1.75	1679	577605	2.0	3	9	2001	0
2964	999000	3	2.75	2830	505166	1.0	4	8	1962	0
9210	350000	4	2.25	1570	499571	1.0	4	7	1972	0
8443	2140000	4	3.75	5150	453895	2.0	3	11	1997	0
7250	950000	4	3.00	3230	438213	2.0	3	9	1999	0
4387	480000	4	3.50	3370	435600	2.0	3	9	2005	0
5073	859000	3	2.50	2920	434728	2.0	4	8	1999	0
2871	302000	2	1.00	900	423838	1.0	5	6	1925	0
2960	474000	2	1.00	1050	403365	1.0	5	6	1905	0
5228	835000	2	3.25	3570	392475	1.0	3	9	1998	0
2757	230000	3	1.00	1530	389126	1.5	4	7	1919	0

Figure 28: Investigation of rows containing Total Area outliers

Total Area:

Unlike the two previous variables, which have a strong correlation, *Total Area* and *Price* have a very weak correlation ($r = 0.1$), which means using *large total area = large price* as a consistency check is methodologically incorrect. Further investigation shall be conducted to determine if the extreme values represent real-world cases. *Total Area* shall be converted to m^2 (square meters) to represent the standard unit for property measurements in Germany and checked against current Berlin property listings and other sources.

```
# measurement unit conversion
SQFT_TO_M2 = 0.092903
df['Total Area (m2)'] = df['Total Area (sqft)'] * SQFT_TO_M2
total_area_outlier_rows = df.query('`Total Area (sqft)` > 19816.12').sort_values(['Total Area (sqft)', 'Year Built'], ascending=False)
display(total_area_outlier_rows.head(10))
```

	Price	Number of Bedrooms	Number of Bathrooms	Living Area (sqft)	Total Area (sqft)	Number of Floors	Property Condition	Property Grade	Year Built	Year Renovated	Total Area (m2)
1719	700000	4	1.00	1300	1651359	1.0	4	6	1920	0	153416.205177
7647	542500	5	3.25	3010	1074218	1.5	5	8	1931	0	99798.074854
7769	855000	4	3.50	4030	1024068	2.0	3	10	2006	0	95138.989404
3949	998000	4	3.25	3770	982998	2.0	3	10	1992	0	91323.463194
4441	790000	2	3.00	2560	982278	1.0	3	8	2004	0	91256.573034
6691	1998000	2	2.50	3900	920423	2.0	3	12	2009	0	85510.057969
7077	1650000	4	3.25	3920	881654	3.0	3	11	2002	0	81908.301562
9714	937500	4	4.00	5545	871200	2.0	3	11	2003	0	80937.093600
4540	550000	3	2.00	3650	843309	2.0	4	7	1991	0	78345.936027
7294	700000	2	1.75	1679	577605	2.0	3	9	2001	0	53661.237315

Figure 29: Evidence of measurement unit conversion

After investigation, the outliers here likely represent multi-family residential blocks, social housing complexes, large-scale residential land parcels, post-war mega-estates/housing areas, villas, or privatized portfolio estates, according to (Landesdenkmalamt Berlin, n.d.), (Senatsverwaltung für Stadtentwicklung, Bauen und W, n.d.), and (Uffer, 2016). As a result, the outliers in this field shall be retained.

This concludes data preprocessing, and the next step of the methodology is analysis. Before proceeding, the fields that contain area measurements (living area and total area) shall be retained strictly using the m² unit in the dataset. After that, a new column of type Boolean shall be created from the *Year Renovated* column, representing whether a house was renovated, to facilitate analysis. Finally, a second exploration shall be performed to show the before and after states of the dataset.

```
df['Living Area (m2)'] = df['Living Area (sqft)'] * SQFT_TO_M2
df.drop(['Living Area (sqft)', 'Total Area (sqft)'], axis=1, inplace=True)
df['Renovation Status'] = df['Year Renovated'] > 0
df.columns

Index(['Price', 'Number of Bedrooms', 'Number of Bathrooms',
       'Number of Floors', 'Property Condition', 'Property Grade',
       'Year Built', 'Year Renovated', 'Total Area (m2)', 'Living Area (m2)',
       'Renovation Status'],
      dtype='object')
```

Figure 30: Final data transformation

Stage 3: Dataset Validation

The goal of this section, as stated above, is to conduct a second exploration of the dataset after it has been transformed to understand its new state.

```
print(f'1. Dataset shape: {df.shape}')
print(f'2. Missing values count: {df.isnull().sum().sum()}')
print(f'3. Duplicate row counts: {df.duplicated().sum()}')
print(f'4. Verifying data types: \n\n{df.dtypes}\n')
print(f'5. New descriptive statistics: \n\n')
display(df.describe())
```

1. Dataset shape:	(9940, 11)
2. Missing values count:	0
3. Duplicate row counts:	0
4. Verifying data types:	
Price	int64
Number of Bedrooms	int64
Number of Bathrooms	float64
Number of Floors	float64
Property Condition	int64
Property Grade	int64
Year Built	int64
Year Renovated	int64
Total Area (m2)	float64
Living Area (m2)	float64
Renovation Status	bool
dtype:	object
5. New descriptive statistics:	

	Price	Number of Bedrooms	Number of Bathrooms	Number of Floors	Property Condition	Property Grade	Year Built	Year Renovated	Total Area (m2)	Living Area (m2)
count	9.940000e+03	9940.000000	9940.000000	9940.000000	9940.000000	9940.000000	9940.000000	9940.000000	9940.000000	9940.000000
mean	5.337212e+05	3.365392	2.063959	1.431137	3.447887	7.594970	1967.245775	91.383300	1491.359372	190.854334
std	3.772516e+05	0.910686	0.765467	0.510786	0.667019	1.162921	27.985419	417.268804	4194.297472	84.651228
min	8.000000e+04	1.000000	0.500000	1.000000	1.000000	3.000000	1900.000000	0.000000	53.140516	35.303140
25%	3.150000e+05	3.000000	1.500000	1.000000	3.000000	7.000000	1950.000000	0.000000	504.277484	130.993230
50%	4.464750e+05	3.000000	2.000000	1.000000	3.000000	7.000000	1969.000000	0.000000	735.327245	175.586670
75%	6.400000e+05	4.000000	2.500000	2.000000	4.000000	8.000000	1990.000000	0.000000	1038.957475	232.257500
max	7.700000e+06	11.000000	8.000000	3.000000	5.000000	13.000000	2015.000000	2015.000000	153416.205177	1119.481150

Figure 31: Dataset exploration after preprocessing

2.0 Data Analysis and Visualization

In the data preprocessing step, univariate analysis was conducted to assess distribution, skewness, and outliers. Here, bivariate analysis, using scatterplots, boxplots, and a kernel density estimate (KDE) overlay plot, is conducted to examine relationships between variables, particularly between *Price* (the primary variable of interest) and other fields. The other fields here are - *Living Area*, *Total Area*, *Property Grade*, *Property Condition*, *Year Built*, and *Renovation Status*, which were selected because they were either explicitly mentioned in the project brief (e.g., *Price and Total Area*) or they showed a meaningful relationship with *Price*.

```
fig, axes = plt.subplots(4, 2, figsize=(17, 24))
fig.suptitle('Housing in Berlin', fontsize=18, fontweight='bold', ha='center', y=1.02)

# 1. KPI Summary Panel
axes[0,0].axis('off')
kpi_text = (
    f"DATASET SUMMARY\n"
    f"{'-'*32}\n"
    f"Total Properties:      {len(df):,}\n"
    f"Renovated:              {df['Renovation Status'].sum():,} ({df['Renovation Status'].mean()*100:.1f}%)\n"
    f"Never Renovated:        {(~df['Renovation Status']).sum():,} ({(~df['Renovation Status']).mean()*100:.1f}%)\n"
    f"MARKET SEGMENTATION\n"
    f"{'-'*32}\n"
    f"Budget (< €315,000):    {(df['Price']<315000).sum():,}\n"
    f"Mid-range:              {(df['Price']>=315000)&(df['Price']<=1127500).sum():,}\n"
    f"Luxury (> €1,127,500):   {(df['Price']>1127500).sum():,}\n"
    f"KEY METRICS\n"
    f"{'-'*32}\n"
    f"Median Price:           €{df['Price'].median():,.0f}\n"
    f"Avg Price per m2:        €{(df['Price']/df['Living Area (m2)']).mean():,.2f}\n"
    f"Most Common Grade:      {df['Property Grade'].mode()[0]}\n"
    f"Property Age Range:      {int(df['Year Built'].min())}-{int(df['Year Built'].max())}"
)
axes[0,0].text(0.05, 0.95, kpi_text, fontsize=16, va='top', ha='left',
    fontfamily='monospace', transform=axes[0,0].transAxes,
    bbox=dict(boxstyle='round', facecolor='whitesmoke', edgecolor='gray'))
axes[0,0].set_title('Key Metrics Summary', fontsize=14)

# 2. Price vs Living Area
sns.regplot(x='Living Area (m2)', y='Price', data=df, ax=axes[0,1],
    scatter_kws={'alpha':0.3, 's':10, 'color':'steelblue'}, line_kws={'color':'darkred', 'ci=None})
axes[0,1].set_title('Price vs Living Area (r = 0.70)', fontsize=11)
axes[0,1].set_xlabel('Living Area (m2)'); axes[0,1].set_ylabel('Price (EUR)')

# 3. Price vs Total Area
sns.regplot(x='Total Area (m2)', y='Price', data=df, ax=axes[1,0],
    scatter_kws={'alpha':0.3, 's':10, 'color':'indianred'}, line_kws={'color':'darkblue', 'ci=None})
axes[1,0].set_title('Price vs Total Area (r = 0.09)', fontsize=11)
axes[1,0].set_xlabel('Total Area (m2)'); axes[1,0].set_ylabel('Price (EUR)')
axes[1,0].set_xlim(0, 250000)

# 4. Price distribution with luxury segment marked
sns.histplot(df['Price'], bins=60, kde=True, ax=axes[1,1])
upper_fence = 1127500
axes[1,1].axvline(upper_fence, color='red', linestyle='--')
axes[1,1].text(upper_fence, axes[1,1].get_ylim()[1]*0.9, ' IQR Upper Fence\n (Luxury threshold)', color='red', fontsize=8)
axes[1,1].set_title('Price Distribution with Luxury Segment Marked', fontsize=11)
axes[1,1].set_xlabel('Price (EUR)'); axes[1,1].set_ylabel('Number of Properties')
```

Figure 32: Code snippet for analysis and visualization 1

```
# 5. Price vs Year Built
sns.regplot(x='Year Built', y='Price', data=df, ax=axes[2,0],
            scatter_kws={'alpha':0.2,'s':10,'color':'seagreen'}, line_kws={'color':'darkred'}, ci=None)
axes[2,0].set_title('Price vs Year Built (r = 0.04)', fontsize=11)
axes[2,0].set_xlabel('Year Built'); axes[2,0].set_ylabel('Price (EUR)')

# 6. Price vs Property Grade (boxplot + median trend line)
sns.boxplot(x='Property Grade', y='Price', data=df, ax=axes[2,1], color='lightsteelblue')
grade_medians = df.groupby('Property Grade')['Price'].median()
axes[2,1].plot(range(len(grade_medians)), grade_medians.values, color='darkred', marker='o', linewidth=2, label='Median Trend')
axes[2,1].set_title('Price vs Property Grade (r = 0.65)', fontsize=11)
axes[2,1].set_xlabel('Property Grade'); axes[2,1].set_ylabel('Price (EUR)')
axes[2,1].legend()

# 7. Price vs Property Condition (boxplot + median trend line)
sns.boxplot(x='Property Condition', y='Price', data=df, ax=axes[3,0], color='lightsalmon')
cond_medians = df.groupby('Property Condition')['Price'].median()
axes[3,0].plot(range(len(cond_medians)), cond_medians.values, color='darkblue', marker='o', linewidth=2, label='Median Trend')
axes[3,0].set_title('Price vs Property Condition (r = 0.04)', fontsize=11)
axes[3,0].set_xlabel('Property Condition'); axes[2,1].set_ylabel('Price (EUR)')
axes[3,0].legend()

# 8. Price vs Renovation Status
sns.kdeplot(df[df['Renovation Status']==False]['Price'], label='Never Renovated', clip=(0, None), fill=True, alpha=0.4, ax=axes[3,1])
sns.kdeplot(df[df['Renovation Status']==True]['Price'], label='Renovated', clip=(0, None), fill=True, alpha=0.4, ax=axes[3,1])
axes[3,1].set_title('Price Distribution by Renovation Status')
axes[3,1].set_xlabel('Price (EUR)'); axes[3,1].legend()

plt.tight_layout()
plt.show()
```

Figure 33: Code snippet for analysis and visualization 2

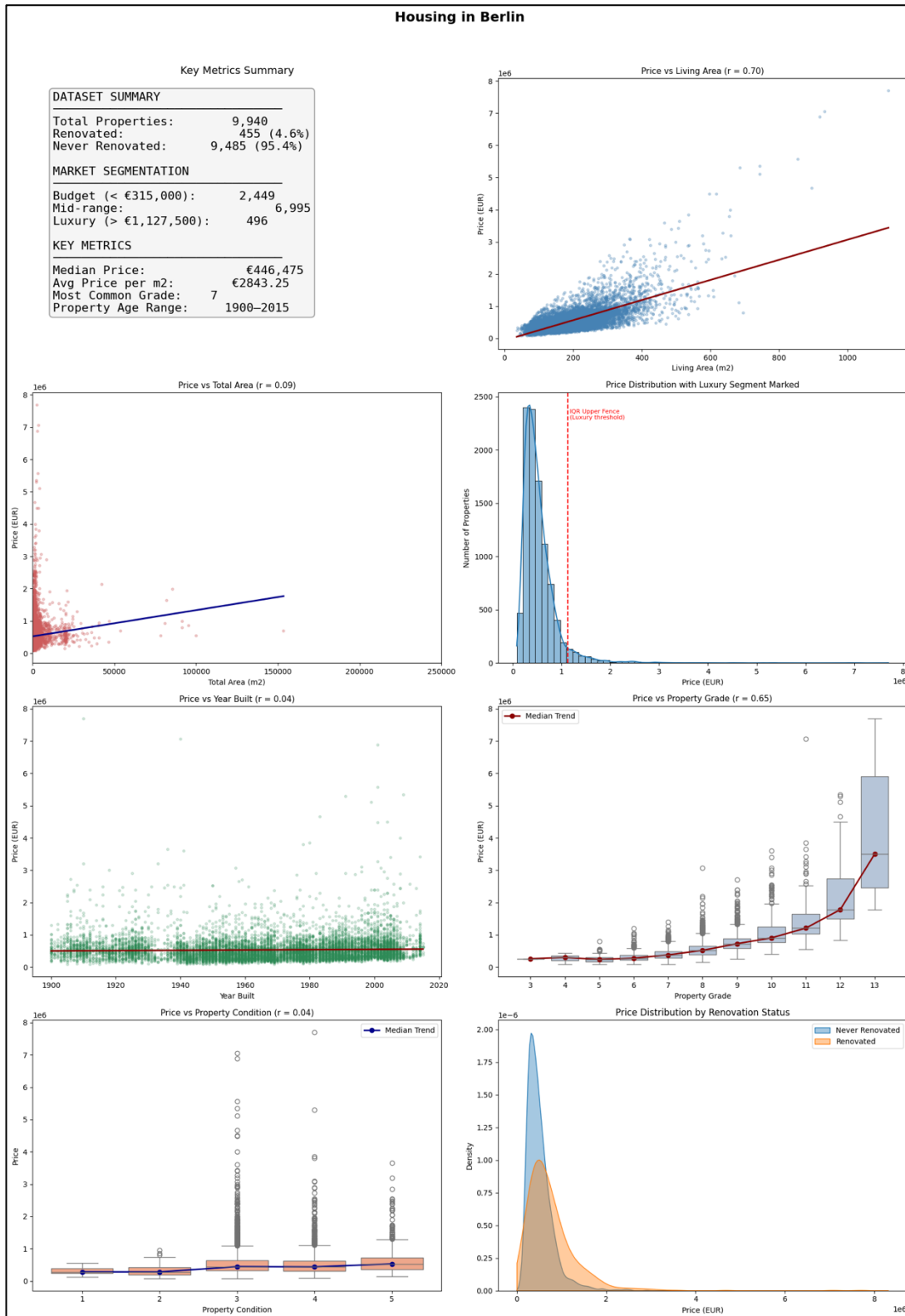


Figure 34: Visual results from analysis

3.0 Reporting

1. Interpretation of Visuals and Justification

- a. The key metrics summary table was included because it is easy to get quick, concise insights.

Key Metrics Summary	
DATASET SUMMARY	
Total Properties:	9,940
Renovated:	455 (4.6%)
Never Renovated:	9,485 (95.4%)
MARKET SEGMENTATION	
Budget (< €315,000):	2,449
Mid-range:	6,995
Luxury (> €1,127,500):	496
KEY METRICS	
Median Price:	€446,475
Avg Price per m2:	€2843.25
Most Common Grade:	7
Property Age Range:	1900–2015

Figure 35: Key metrics summary

- b. A scatterplot is appropriate to use when two continuous variables are involved. In Figure 34, the scatterplots were accompanied by trendlines to make correlation visually understandable.
 - i. **Price vs. Living Area** shows a clear upward-sloping cloud with its spread widening at higher values, indicating that properties with larger living areas tend toward higher prices.

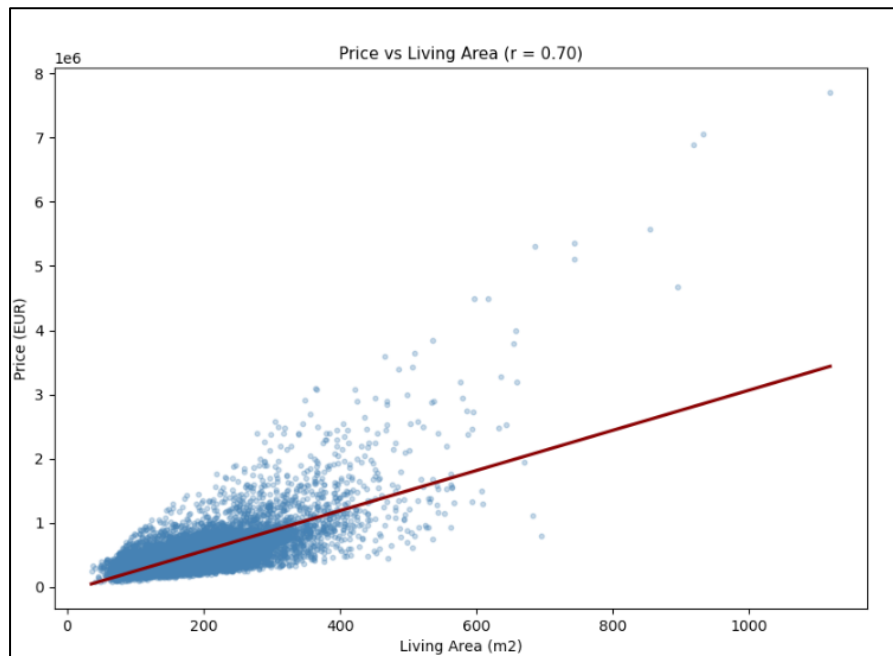


Figure 36: Price vs Living Area

- ii. **Price vs. Total Area** shows a shapeless scatter, which confirms its near-zero correlation, implying total area does not have an effect on house pricing.

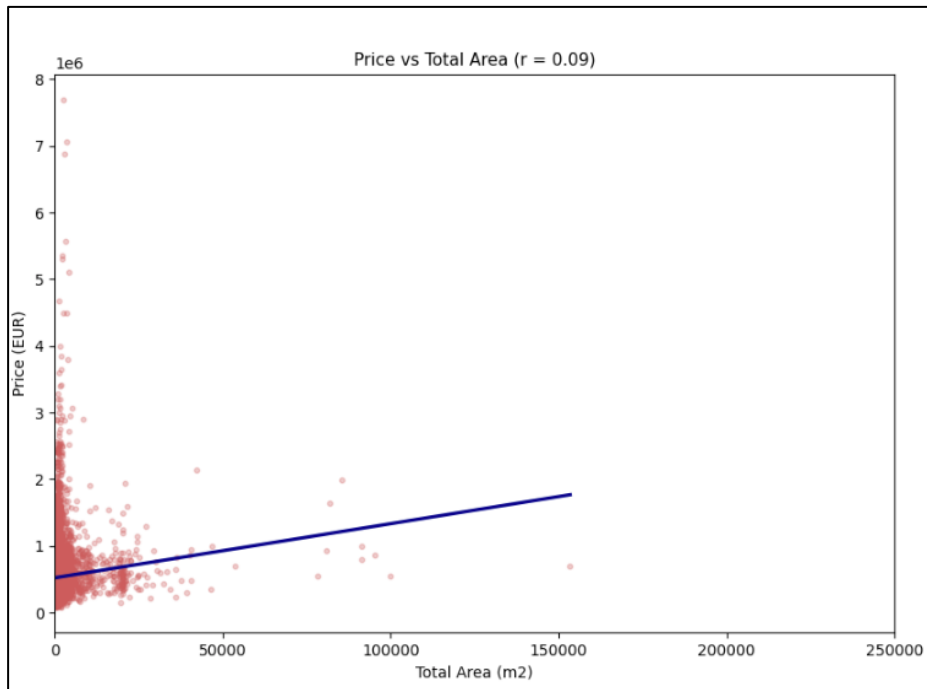


Figure 37: Price vs Total Area

- iii. **Price vs. Year Built** shows a flat pattern, indicating that construction year does not affect price.

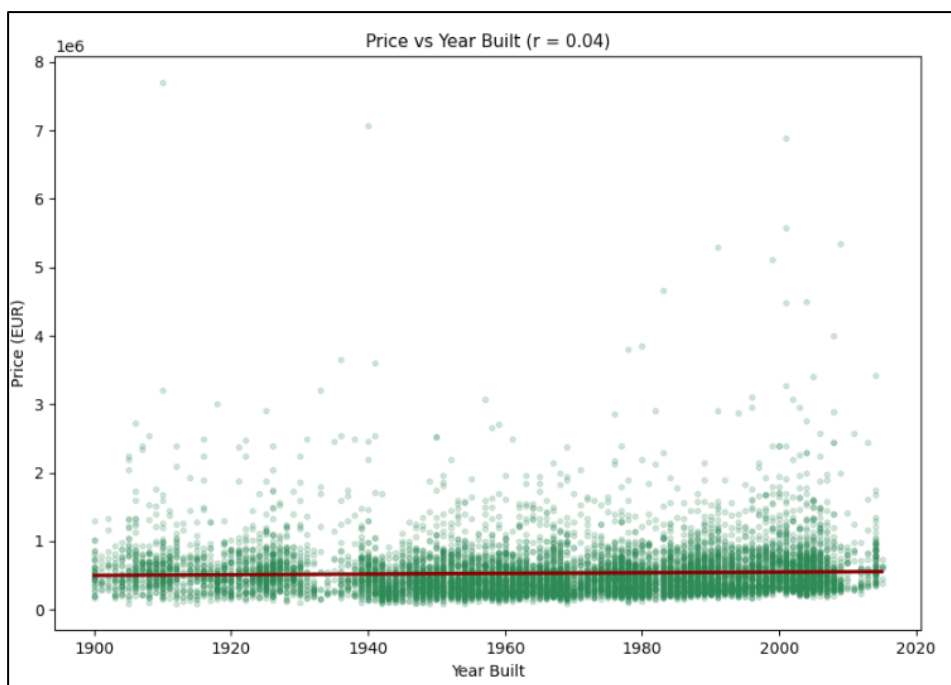


Figure 38: Price vs Year Built

- c. Apart from identifying outliers, Boxplots are used to summarize the distribution of numerical data and compare groups. Here, *Price* is compared with *Property Grade* as well as *Property Condition*, and the median trendline was used because the mean results in distortion due to extreme values, while the mode does not often exist for a variable like Price, which has values differing in small rates.

- i. **Price vs. Property Grade** shows a monotonic staircase, with price rising consistently, confirming its strong correlation.

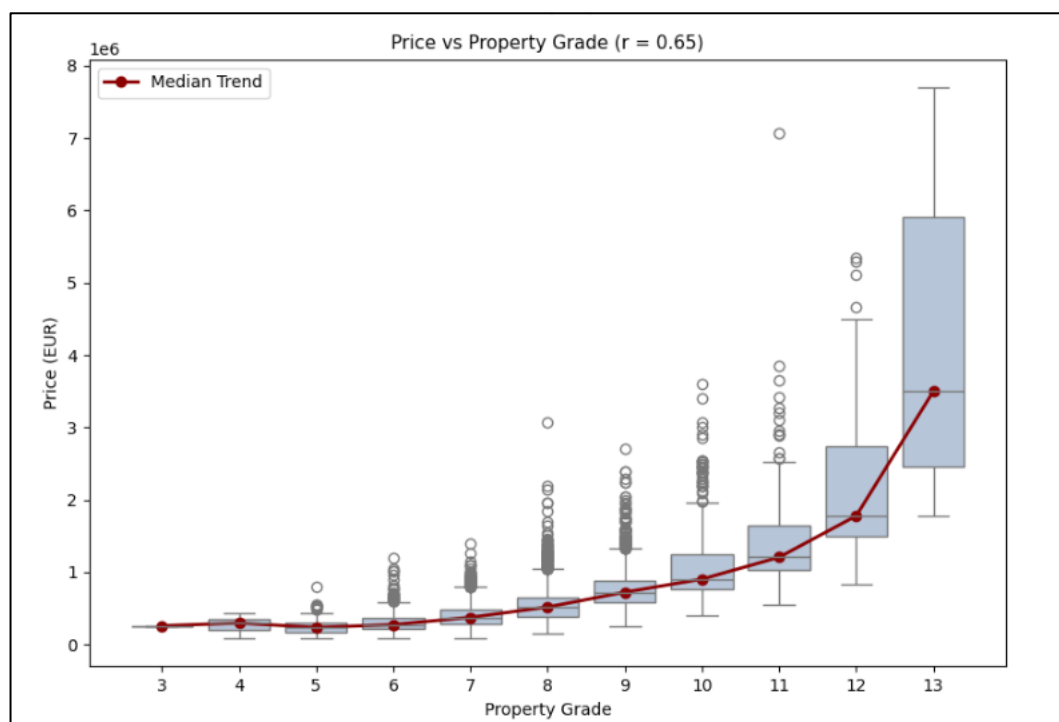


Figure 39: Price vs Property Grade

- ii. **Price vs. Property Condition** shows no consistent pattern, which is in line with its negligible correlation.

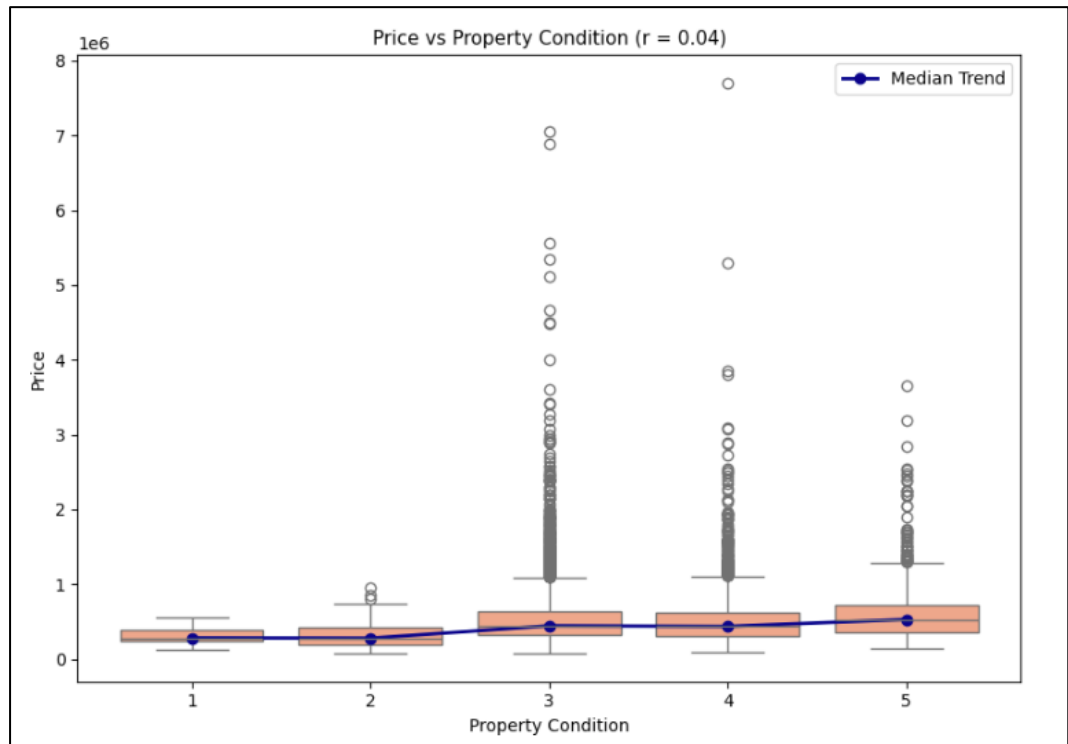


Figure 39: Price vs Property Condition

- d. Histograms are used to display the distribution of numerical data, specifically to see frequency, shape, and spread. A **price histogram** was used to show the count of houses that fall into the different pricing categories. A line is also used to mark the luxury segment of the properties.

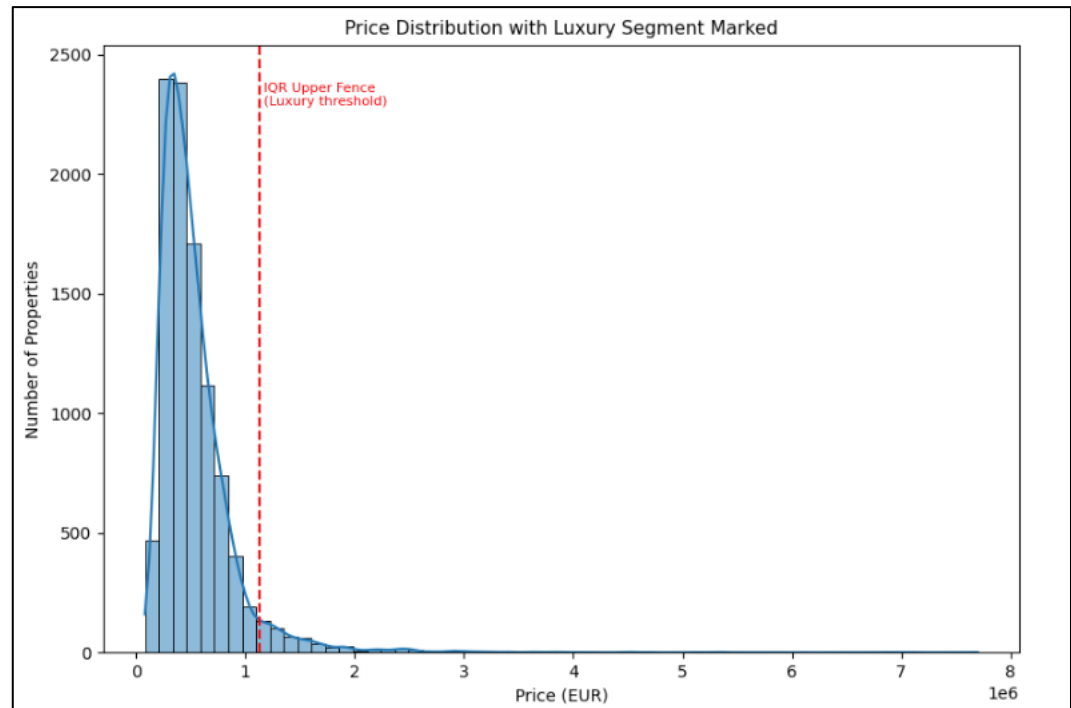


Figure 40: Price distribution with luxury threshold

- e. A KDE overlay plot is used to compare the shapes of distributions across groups. Here, **pricing** is compared against **renovation status**. Both curves peak in the lower range, but the "Never Renovated" curve peaks higher and sharper compared to "Renovated," whose peak is shorter and more shifted to the right. This shows that never-renovated properties tend to be cheaper on average than renovated ones.

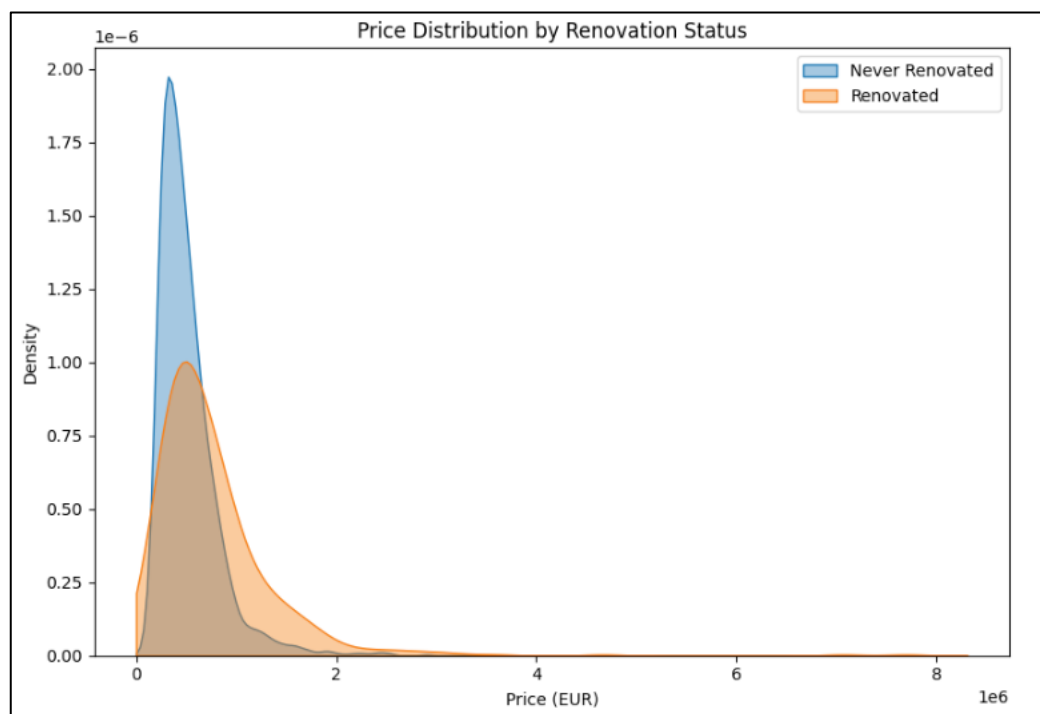


Figure 41: Price distribution by Renovation Status

2. Key Findings and Recommendations

Table 5: Insights and Recommendations

	Key Finding	Recommendation	Additional Notes
1	Living Area ($r = 0.70$) and Property Grade ($r = 0.65$) are the most dominant drivers of price.	They should be prioritized as the core inputs for predicting future housing trends in Berlin.	The insight from property grade influencing price can also be used to advise new housing construction or major renovation projects. Additionally, it is worth noting that properties with large living areas, high grade, and renovation done (will be further discussed below) each individually show strong association with higher prices, suggesting that their combination could correspond with premium pricing, although this was not tested as a combined effect.
2	Renovation is associated with higher pricing, but evidence is limited.	Before it is used as a confident pricing factor, a larger sample of renovated properties should be investigated to strengthen the reliability of this finding.	
3	Total Area ($r = 0.09$), Property Condition ($r = 0.04$), and Year Built ($r = 0.04$) are weak and unreliable predictors of price.	They should not be relied on in isolation for predicting future housing trends, as their inclusion in a model would most likely add noise.	
4	The Berlin housing market, according to this dataset, contains a luxury segment.	Before prediction is done, these properties should be segmented into "typical" and "luxury," as a single blended average price misinterprets both segments and could in turn mislead trend forecasting.	
5	The dataset is dated over a decade.	Given the time that this project is being conducted (2026) and the time range of the dataset (1900 – 2015), this dataset should be updated with more recent property listings for more accurate and time-relevant results when forecasting.	
6	The columns <i>Living Area (sqft)</i> (<i>sqft_living</i>) and <i>Living Area (sqft) 2</i> (<i>living_area_sqft</i>) are strongly related ($r = 0.75$) but distinct.	The purpose of the <i>Living Area (sqft) 2</i> variable should be clarified.	The <i>Living Area (sqft)</i> field should be retained as the standard working variable during predictive modelling because of its stronger correlation with <i>Price</i> and thus more relevance to the project.
7	The <i>Total Area</i> column contains many (1185) extreme values, ranging from 50,000 m ² upward, representing properties such as portfolio estates rather than regular housing.	A field such as " <i>Property Type</i> " should be added so properties can be validated across the correct category.	

3. Limitations

1. The dataset shows only year built and year renovated. It does not directly show how prices change over time, which limits forecasting pricing trends.
2. Property conditions 1 and 2 constitute 12 and 84 properties, respectively, and the renovated group, which makes up 4.6% of the dataset, is comparatively small, thus reducing confidence in findings tied to them.

```
print(f'Renovated houses % {(len(df.query(`Renovation Status` == True)) / len(df)) * 100}%')
print(f'Count of Property Condition 1: {len(df.query(`Property Condition` == 1))}')
print(f'Count of Property Condition 2: {len(df.query(`Property Condition` == 2))}')
Renovated houses % 4.577464788732395
Count of Property Condition 1: 12
Count of Property Condition 2: 84
```

Figure 42: Proof of claims in (2)

3. As per the brief, no model was applied to properly investigate assumed/suggested possible combined effects (large living area + high property grade + renovated property = higher pricing).
4. Findings are specific to this dataset and do not generalize to the Berlin housing market without further investigation/validation and a more recent sample.

Reflective Report

This project involved conducting EDA on the *BerlinHousing4049.csv* dataset comprising 9999 property records. While working on this project, I followed the steps in the data analysis lifecycle. Given that the dataset was already provided, I started by doing an initial data exploration to understand the state of the dataset; this included getting summary statistics, confirming data types and the shape, and checking for duplicates and missing data. After completing this, I found out it contained two duplicates and 0.62% of missing data, as well as data types that did not fully correspond to the original dataset, which made the data semantically incorrect; for example, the *built* column contained years as floating-point values such as *2015.0*. I then proceeded to clean the data, which involved identifying and handling duplicates, missing data, outliers, wrongly formatted data, and incorrect data. For the outliers, I used the 1.5 IQR method since the data points in the selected fields were not normally distributed, and for missing data, I deleted the records that contained them because they constituted less than 1% of the entire dataset, meaning it would not significantly affect the analysis. I also changed the data types of the columns where appropriate and performed logical consistency checks to make sure the dataset was free of errors, such as checking that it did not contain any price value less than 0. When this was done, I did a second data exploration to compare the before and after states of the dataset before moving into analysis and visualization.

The key insights I got from my analysis were that living area and property grade are key factors influencing price in this dataset and therefore should be used as core inputs when predicting future housing trends, while total area is not. Also, I discovered that the two living area columns (*sqft_living* and *living_area_sqft*) have a strong correlation (Pearson correlation coefficient of 0.75); however, they are distinct, and I could not determine the main reason for their difference, although renovation status proved to be a minor contributing factor. While conducting my analysis, I used visualizations such as histograms, boxplots, scatterplots, and KDE overlay plots to gain a clearer and visual understanding of my findings.

The main challenge I faced here was handling outliers in the total area column; several rows exceeded 50,000 m², which was far beyond any comparable listing I could find for Berlin

properties listed on rental websites. After discussing with my lecturer and finding a few resources that proved Berlin actually has residential properties with extremely large total areas (50,000 m² and above), I was able to finally conclude on retaining these outliers in the dataset, as they represented real-world data and also constituted a large percentage of the dataset (11.92%).

Aside from prioritizing living area and property grade in future predictive work, I also recommend clearly outlining the significance of both living area columns and categorizing properties based on their plot size (total area) and luxury group (pricing of houses) to mitigate the existence of outliers and overall uncertainty when conducting analysis or building a model.

In conclusion, this project strengthened my analytical skills and reinforced the importance of excellent data preprocessing and research, especially when investigating topics you have little mastery of, which, in my case, is real estate. For my next steps, I intend to increase the sample size of this dataset to reflect recent property listings while addressing the limitations and implementing the recommendations stated in the previous two sections before building a predictive model that can forecast housing market trends in Berlin.

References

- Bruce, P., Bruce, A. & Gedeck, P., 2020. *Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python*. Sebastopol(Carlifornia): O'Reilly Media, Inc..
- Dudáš, A., 2024. *Graphical representation of data prediction potential: correlation graphs and correlation chains*, Banská Bystrica: Springer Nature.
- Fannie Mae, 2024. *Selling Guide*. [Online]
Available at: <https://selling-guide.fanniemae.com/>
[Accessed 16 July 2026].
- George, D. & Mallery, P., 2019. *IBM SPSS Statistics 26 Step by Step: A Simple Guide and Reference*. 16th ed. New York: Routledge.
- Hammer, B., Virgili, E. & Bilotta, F., 2023. *Evidence-based literature review: De-duplication a cornerstone for quality*. [Online]
Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC10789108/#_ci93
[Accessed 17 July 2026].
- Heymans, M. W. & Twisk, J. W., 2022. *Handling missing data in clinical research*. [Online]
Available at: <https://www.sciencedirect.com/science/article/pii/S0895435622002189>
[Accessed 17 July 2026].
- Indiana Department of Local Government Finance, 2021. *Real Property Assessment Guidelines*, Indianapolis: Indiana Department of Local Government Finance.
- Kent State University Libraries, 2026. *SPSS Tutorials: Pearson Correlation*. [Online]
Available at: <https://libguides.library.kent.edu/SPSS/PearsonCorr>
[Accessed July 2026].
- Landesdenkmalamt Berlin, n.d.. *Siedlungen der Berliner Moderne*. [Online]
Available at: <https://www.berlin.de/landesdenkmalamt/welterbe/siedlungen-der-berliner-moderne/>
[Accessed July 2026].
- McKinney, W., 2022. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. Sebastopol(Carlifornia): O'Reilly Media.
- National Association of REALTORS®, 2020. *Pricing Strategies: Mastering the CMA (PSA Student Manual)*, Chicago: Center for Specialized REALTOR® Education.
- Pandas Contributors, 2025. *Nullable integer data type*. [Online]
Available at: https://pandas.pydata.org/docs/user_guide/integer_na.html
[Accessed July 2026].
- Senatsverwaltung für Stadtentwicklung, Bauen und W, n.d.. *Geoportal Berlin / Geoinformation*. [Online]
Available at: <https://www.berlin.de/sen/stadt/stadtdaten/geoinformation/>
[Accessed July 2026].

Sharma, R. & Garg, P., 2024. Fundamentals of Data Analytics and Lifecycle. In: *Data Analytics and Machine Learning*. Singapore: Springer.

Sharou, A. et al., 2021. Towards a Better Understanding of Noise in Natural Language Processing. In: *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*. "Held Online": INCOMA Ltd..

Uffer, S., 2016. The uneven development of Berlin's housing provision: Institutional investment and its consequences on the city and its tenants. *Urban Studies*, 53(12), p. 2360–2376.

Wickham, H., 2014. Tidy Data. *Journal of Statistical Software*, 59(10), pp. 1-23.

Appendix

Jupyter Notebook URL

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

df = pd.read_csv('/content/BerlinHousing4049.csv')
df

df.info()
df.describe()

columns_with_missing_values = len(df.columns[df.isnull().any()])
print('Columns with missing data:', columns_with_missing_values)

rows_with_missing_values = df.isnull().any(axis=1).sum()
print('Rows with missing data:', rows_with_missing_values)

missing_values = df.isnull().sum().sum()
missing_values_percentage = (missing_values / df.shape[0]) * 100
print(f'Percentage of missing values: {round(missing_values_percentage, 2)}%')

missing_values_percentage_per_column = round((df.isnull().sum() / df.shape[0]) * 100, 2)
display(missing_values_percentage_per_column.sort_values(ascending=False))

duplicates = df.duplicated().sum()
print("\nDuplicates count:", duplicates)

print('Row count before duplicate removal:', df.shape[0])
df.drop_duplicates(inplace=True)
print('Row count after duplicate removal:', df.shape[0])
```

```
print('% missing values before Listwise Deletion: 0.62%')
print('Row count before Listwise Deletion:', df.shape[0])
df.dropna(inplace=True)
print('Row count after Deduplication & Listwise Deletion:', df.shape[0])
print(f'% missing values after Listwise Deletion: {round((df.isnull().sum().sum()/ df.shape[0]) * 100, 2)}%')
```

1. Changing data type

```
print('Data types transformation')
int_columns = [
    'price',
    'bedrooms',
    'sqft_living',
    'sqft_total',
    'condition',
    'built',
    'living_area_sqft',
]
for column in int_columns:
    df[column] = df[column].astype('int64')

display(df.dtypes)
```

#2. Renaming columns

```
print("\nColumn renaming\n")
df.rename(columns={
    'price': 'Price',
    'bedrooms': 'Number of Bedrooms',
    'bathrooms': 'Number of Bathrooms',
    'sqft_living': 'Living Area (sqft)',
    'sqft_total': 'Total Area (sqft)',
    'floors': 'Number of Floors',
    'condition': 'Property Condition',
    'grade': 'Property Grade',
    'built': 'Year Built',
```

```
'renovated': 'Year Renovated',
'living_area_sqft': 'Living Area (sqft) 2',
}, inplace=True)
df.columns
```

```
living_area_columns = df[['Living Area (sqft)', 'Living Area (sqft) 2', 'Year Renovated']].copy()
living_area_columns
```

step 1: get the % of identical count

```
identical_count = (living_area_columns['Living Area (sqft)'] == living_area_columns['Living Area (sqft)
2']).sum()
differing_count = (living_area_columns['Living Area (sqft)'] != living_area_columns['Living Area (sqft)
2']).sum()
```

```
print(fIdentical count: {identical_count})
print(fDiffering count: {differing_count})
print(fPercentage of Identical Count: {identical_count / len(living_area_columns) * 100:.2f}%)
```

step 2: distribution of the gap

```
living_area_columns['Gap'] = (living_area_columns['Living Area (sqft)'] - living_area_columns['Living
Area (sqft) 2'])
round(living_area_columns['Gap'].describe(), 2)
```

step 3: testing whether gap relates to renovation status

```
living_area_columns['Gap Abs'] = living_area_columns['Gap'].abs()
renovated = living_area_columns[living_area_columns['Year Renovated'] > 0]
never_renovated = living_area_columns[living_area_columns['Year Renovated'] == 0]
```

```
print(fRenovated properties -> mean absolute gap: {round(renovated["Gap Abs"].mean(), 2)} sqft
(n={len(renovated)}))
print(fNever Renovated properties -> mean absolute gap: {round(never_renovated["Gap Abs"].mean(),
2)} sqft (n={len(never_renovated)}))
```

```
print(fRenovated % identical: {(renovated['Living Area (sqft)'] == renovated['Living Area (sqft)
2']).mean() * 100:.2f}%)
```

```
print(f'Never Renovated % identical: {(never_renovated["Living Area (sqft)"] == never_renovated["Living Area (sqft) 2"]).mean() * 100:.2f}%')
```

```
# 1: full pearson correlation matrix across all columns
```

```
corr = df.corr(method='pearson')
```

```
round(corr, 2)
```

```
# displaying matrix as heatmap
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(round(corr, 1), annot=True, center=0)
```

```
plt.title('Pearson Correlation Heatmap')
```

```
plt.show()
```

```
df.drop('Living Area (sqft) 2', axis=1, inplace=True)
```

```
df.columns
```

```
# part 1
```

```
range_checks = {
```

```
    'Price <= 0': (df['Price'] <= 0).sum(),
```

```
    'Bedrooms < 0': (df['Number of Bedrooms'] < 0).sum(),
```

```
    'Bathrooms <= 0': (df['Number of Bathrooms'] <= 0).sum(),
```

```
    'Floors <= 0': (df['Number of Floors'] <= 0).sum(),
```

```
    'Total Area (sqft) <= 0': (df['Total Area (sqft)'] <= 0).sum(),
```

```
    'Living Area (sqft) <= 0': (df['Living Area (sqft)'] <= 0).sum(),
```

```
}
```

```
for rule, count in range_checks.items():
```

```
    print(f'{rule}: {count} violation(s)')
```

```
### part 2
```

```
space_violation = df['Living Area (sqft)'] > (df['Total Area (sqft)'] * df['Number of Floors'])
```

```
print(f'Adjusted rule violations: {space_violation.sum()} out of {len(df)} rows')
```

```
print(f'Violation %: {space_violation.sum()/len(df) * 100:.2f}%')
```

```
display(df[space_violation])
```

```
# part 3
```

```
max_year_built = df['Year Built'].max()
```

```

min_year_built = df['Year Built'].min()
print(fMax Year Built: {max_year_built})
print(fMin Year Built: {min_year_built})

renovated_before_built = df[(df['Year Renovated'] < df['Year Built']) & (df['Year Renovated'] > 0)]
renovated_out_of_dataset_timeframe = df[((df['Year Renovated'] > max_year_built) | (df['Year
Renovated'] < min_year_built)) & (df['Year Renovated'] > 0)]
print(fRenovated before built: {len(renovated_before_built)} violation(s))
print(fRenovated out of dataset timeframe: {len(renovated_out_of_dataset_timeframe)} violation')

for item in df[space_violation].index:
    total_area = df.loc[item, 'Total Area (sqft)']
    number_of_floors = df.loc[item, 'Number of Floors']
    living_area = df.loc[item, 'Living Area (sqft)']

    # Recalculate difference and percentage for the current row
    difference = (total_area * number_of_floors) - living_area
    percentage = (difference / (total_area * number_of_floors)) * 100
    print(fRow: {item} Excess: {abs(difference):.0f} sqft Percentage: {percentage:.2f}%)

df.drop(df[space_violation].index[0:2], inplace=True)
df.shape[0]

# step 1: calculate distribution of data
variables = ['Price', 'Total Area (sqft)', 'Living Area (sqft)',]
for variable in variables:
    print(f----- {variable} -----')
    print(fMean: {round(df[variable].mean(), 2)})
    print(fMedian: {round(df[variable].median(), 2)})
    print(fMode: {round(df[variable].mode()[0], 2)})
    print(fStd. Dev: {round(df[variable].std(), 2)})
    print(fSkewness: {round(df[variable].skew(), 2)})
    print(fKurtosis: {round(df[variable].kurtosis(), 2)})
    print()

```

```
fig, axes = plt.subplots(len(variables), 1, figsize=(6, 4 * len(variables)))
```

```
for i, variable in enumerate(variables):
    sns.histplot(df[variable], bins=30, kde=True, ax=axes[i])
    axes[i].set_title(f'Histogram of {variable}')
```

```
plt.tight_layout()
plt.show()
```

```
def iqr_outliers(series: pd.Series):
    """
    Calculates outliers for a given variable
    param series: the variable for which the outlier is to be calculated
    """
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - (1.5 * iqr)
    upper_bound = q3 + (1.5 * iqr)
    outliers = series[(series < lower_bound) | (series > upper_bound)]
    return q1, q3, iqr, lower_bound, upper_bound, outliers
```

```
for variable in variables:
    q1, q3, iqr, lower_bound, upper_bound, outliers = iqr_outliers(df[variable])
    print(f'----- {variable} -----')
    print(f'Q1: {round(q1, 2)}, Q3: {round(q3, 2)}, IQR: {round(iqr, 2)}')
    print(f'Lower Bound: {round(lower_bound, 2)}, Upper Bound: {round(upper_bound, 2)}')
    print(f'Number of Outliers: {len(outliers)} Percentage of Outliers: {len(outliers)/len(df) * 100:.2f}%')
    print()
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
```

```
for i, variable in enumerate(variables):
    q1 = df[variable].quantile(0.25)
    q3 = df[variable].quantile(0.75)
```

```

iqr = q3 - q1
upper = q3 + 1.5 * iqr
lower = q1 - 1.5 * iqr
n_out = ((df[variable] < q1 - 1.5*iqr) | (df[variable] > upper)).sum()

sns.boxplot(y=df[variable], ax=axes[i])
axes[i].set_title(f'{variable}\nOutliers: {n_out} ({n_out/len(df)*100:.2f}%)', fontsize=10)
axes[i].axhline(upper, color='red', linestyle='--', linewidth=1)
axes[i].text(0.05, upper, f'Upper Fence: {upper:,.0f}', color='red', fontsize=8, va='bottom')
axes[i].text(0.05, lower, f'Lower Fence: {lower:,.0f}', color='red', fontsize=8, va='bottom')
plt.tight_layout()
plt.show()

# Variable 1: Price
print('Rows containing outliers for the price column:')
price_outlier_rows = df.query('Price > 1127500')
display(price_outlier_rows.head(20).sort_values(by='Property Grade', ascending=True))

# variable 2: Living Area (sqft)
print('Rows containing outliers for the Living Area column:')
living_area_outlier_rows = df.query('Living Area (sqft)' > 4135').sort_values('Living Area (sqft)',
ascending=False)
display(living_area_outlier_rows.head(20))

# Variable 3: Total Area (sqft)
print('Rows containing outliers for the Total Area column:')
total_area_outlier_rows = df.query('Total Area (sqft)' > 19816.12').sort_values('Total Area (sqft)',
ascending=False)
display(total_area_outlier_rows.head(20))

# measurement unit conversion
SQFT_TO_M2 = 0.092903
df['Total Area (m2)'] = df['Total Area (sqft)'] * SQFT_TO_M2
total_area_outlier_rows = df.query('Total Area (sqft)' > 19816.12').sort_values(['Total Area (sqft)', 'Year
Built'], ascending=False)

```

```
display(total_area_outlier_rows.head(10))
```

```
df['Living Area (m2)'] = df['Living Area (sqft)'] * SQFT_TO_M2
df.drop(['Living Area (sqft)', 'Total Area (sqft)'], axis=1, inplace=True)
df['Renovation Status'] = df['Year Renovated'] > 0
df.columns
```

```
print(f1. Dataset shape: {df.shape})
print(f2. Missing values count: {df.isnull().sum().sum()})
print(f3. Duplicate row counts: {df.duplicated().sum()})
print(f4. Verifying data types: \n\n{df.dtypes}\n')
print(f5. New descriptive statistics: \n\n')
display(df.describe())
```

```
fig, axes = plt.subplots(4, 2, figsize=(17, 24))
fig.suptitle('Housing in Berlin', fontsize=18, fontweight='bold', ha='center', y=1.02)
```

1. KPI Summary Panel

```
axes[0,0].axis('off')
kpi_text = (
    f"DATASET SUMMARY\n"
    f"{'-'*32}\n"
    f"Total Properties: {len(df):,}\n"
    f"Renovated: {df['Renovation Status'].sum():,} ( {df['Renovation Status'].mean()*100:.1f}%)\n"
    f"Never Renovated: {(~df['Renovation Status']).sum():,} ( {(~df['Renovation Status']).mean()*100:.1f}%)\n\n"
    f"MARKET SEGMENTATION\n"
    f"{'-'*32}\n"
    f"Budget (< €315,000): {(df['Price']<315000).sum():,}\n"
    f"Mid-range: {((df['Price']>=315000)&(df['Price']<=1127500)).sum():,}\n"
    f"Luxury (> €1,127,500): {(df['Price']>1127500).sum():,}\n\n"
    f"KEY METRICS\n"
    f"{'-'*32}\n"
    f"Median Price: €{df['Price'].median():.0f}\n"
    f"Avg Price per m2: €{(df['Price']/df['Living Area (m2)']).mean():.2f}\n"
```



```
f"Most Common Grade: {df['Property Grade'].mode()[0]}\n"
f"Property Age Range: {int(df['Year Built'].min())}–{int(df['Year Built'].max())}"
axes[0,0].text(0.05, 0.95, kpi_text, fontsize=16, va='top', ha='left',
               fontfamily='monospace', transform=axes[0,0].transAxes,
               bbox=dict(boxstyle='round', facecolor='whitesmoke', edgecolor='gray'))
axes[0,0].set_title('Key Metrics Summary', fontsize=14)

# 2. Price vs Living Area
sns.regplot(x='Living Area (m2)', y='Price', data=df, ax=axes[0,1],
            scatter_kws={'alpha':0.3,'s':10,'color':'steelblue'}, line_kws={'color':'darkred'}, ci=None)
axes[0,1].set_title('Price vs Living Area (r = 0.70)', fontsize=11)
axes[0,1].set_xlabel('Living Area (m2)'); axes[0,1].set_ylabel('Price (EUR)')

# 3. Price vs Total Area
sns.regplot(x='Total Area (m2)', y='Price', data=df, ax=axes[1,0],
            scatter_kws={'alpha':0.3,'s':10,'color':'indianred'}, line_kws={'color':'darkblue'}, ci=None)
axes[1,0].set_title('Price vs Total Area (r = 0.09)', fontsize=11)
axes[1,0].set_xlabel('Total Area (m2)'); axes[1,0].set_ylabel('Price (EUR)')
axes[1,0].set_xlim(0, 250000)

# 4. Price distribution with luxury segment marked
sns.histplot(df['Price'], bins=60, kde=True, ax=axes[1,1])
upper_fence = 1127500
axes[1,1].axvline(upper_fence, color='red', linestyle='--')
axes[1,1].text(upper_fence, axes[1,1].get_ylim()[1]*0.9, 'IQR Upper Fence\n(Luxury threshold)',
               color='red', fontsize=8)
axes[1,1].set_title('Price Distribution with Luxury Segment Marked', fontsize=11)
axes[1,1].set_xlabel('Price (EUR)'); axes[1,1].set_ylabel('Number of Properties')

# 5. Price vs Year Built
sns.regplot(x='Year Built', y='Price', data=df, ax=axes[2,0],
            scatter_kws={'alpha':0.2,'s':10,'color':'seagreen'}, line_kws={'color':'darkred'}, ci=None)
axes[2,0].set_title('Price vs Year Built (r = 0.04)', fontsize=11)
axes[2,0].set_xlabel('Year Built'); axes[2,0].set_ylabel('Price (EUR)')
```

6. Price vs Property Grade (boxplot + median trend line)

```
sns.boxplot(x='Property Grade', y='Price', data=df, ax=axes[2,1], color='lightsteelblue')
grade_medians = df.groupby('Property Grade')['Price'].median()
axes[2,1].plot(range(len(grade_medians)), grade_medians.values, color='darkred', marker='o', linewidth=2,
label='Median Trend')
axes[2,1].set_title('Price vs Property Grade (r = 0.65)', fontsize=11)
axes[2,1].set_xlabel('Property Grade'); axes[2,1].set_ylabel('Price (EUR)')
axes[2,1].legend()
```

7. Price vs Property Condition (boxplot + median trend line)

```
sns.boxplot(x='Property Condition', y='Price', data=df, ax=axes[3,0], color='lightsalmon')
cond_medians = df.groupby('Property Condition')['Price'].median()
axes[3,0].plot(range(len(cond_medians)), cond_medians.values, color='darkblue', marker='o', linewidth=2,
label='Median Trend')
axes[3,0].set_title('Price vs Property Condition (r = 0.04)', fontsize=11)
axes[3,0].set_xlabel('Property Condition'); axes[2,1].set_ylabel('Price (EUR)')
axes[3,0].legend()
```

8. Price vs Renovation Status

```
sns.kdeplot(df[df['Renovation Status']==False]['Price'], label='Never Renovated', clip=(0, None), fill=True,
alpha=0.4, ax=axes[3,1])
sns.kdeplot(df[df['Renovation Status']==True]['Price'], label='Renovated', clip=(0, None), fill=True,
alpha=0.4, ax=axes[3,1])
axes[3,1].set_title('Price Distribution by Renovation Status')
axes[3,1].set_xlabel('Price (EUR)'); axes[3,1].legend()
```

```
plt.tight_layout()
plt.show()
```

```
print(f'Renovated houses % {(len(df.query("Renovation Status" == True)) / len(df)) * 100}')
print(f'Count of Property Condition 1: {len(df.query("Property Condition" == 1))}')
print(f'Count of Property Condition 2: {len(df.query("Property Condition" == 2))}')
```